

MEDIATEK

MT8676 SCP

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Huayu Zong	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	4
List of Tables	5
1 SCP Overview	6
1.1 Hardware Architecture	6
1.2 Software Architecture	8
1.3 Summary	9
2 Source Tree	10
2.1 Little Kernel Bootloader	10
2.2 Linux Kernel Driver	10
2.3 FreeRTOS Tree	10
3 Build System	11
3.1 Configuration Files	11
3.1.1 Little Kernel Bootloader	11
3.1.2 Linux Kernel	11
3.1.3 FreeRTOS	11
3.2 Build Commands	12
3.2.1 Standalone	12
3.2.2 With AOSP Hierarchy	12
3.3 Image Layout	12
4 Boot Sequence	13
5 Inter Processor Interrupt (IPI)	14
5.1 Usage on the SCP Side	15
5.2 Usage on the Kernel Side	22
6 DRAM Access Procedure	25
6.1 Reserving Memory in Linux Kernel	26
6.1.1 Register an User in Shared Memory	26
6.1.2 Extend Shared Memory Size	27
6.2 Get Reserved Memory by ID	28
6.3 Get Reserved DRAM Region on SCP Side	28
6.4 Remap DRAM Address from AP View to SCP View	29
6.5 Request System Bus and DRAM	29
7 Drivers Guide	31
7.1 Driver Initiation	31
7.2 Add a New Driver	31
7.3 Interrupt	32
7.3.1 IRQ Registration	32
7.3.2 Enable IRQ	33
7.3.3 Disable IRQ	33
7.3.4 Wakeup Source Registration	34
7.4 Locks	35
7.5 DMA	36

7.6	Hardware Semaphore	37
7.7	GPIO & EINT	38
7.7.1	GPIO Usage	39
7.7.2	EINT Usage	40
8	Debug Methods	41
8.1	PRINTF_* Usage	41
8.2	Mobile Log	41
8.3	UART	42
8.4	ADB Logcat	42
8.5	Exception Log Analysis	44
8.5.1	Trace Buffer	46
8.6	Coredump	47
8.6.1	LLDB Basic Commands	49
8.7	Performance Evaluation and Flight Record	52
8.7.1	Performance Budget	52
8.7.2	Flight Record	54
9	Address Sanitizer	60
9.1.1	ASAN Usage	60
9.1.2	How to Move Code to DRAM	62
10	Appendix	64
10.1	Frequently Asked Questions/Troubleshooting	64
10.1.1	Malloc Failure	64
10.1.2	Unaligned Access	65
10.2	Code Size Limitation	66
10.2.1	Code Size Tool Usage	66
10.3	Scp_Region_Info Structure	68
10.4	SCP Recovery	69
10.4.1	Recovery Behavior	69
10.4.2	Recovery Notify Flow	69
11	Q&A List from Users	71
11.1	How to Enlarge DRAM Region Code	71
	Exhibit 1 Terms and Conditions	72

List of Figures

Figure 1-1.	MTK 23P SCP architecture	7
Figure 1-2.	MTK MT6897 SCP software architecture	8
Figure 4-1.	SCP boot up flow overview	13
Figure 5-1.	Tinysys common IPI architecture	14
Figure 8-1.	MTKLogger	41
Figure 8-2.	UART setting	42
Figure 8-3.	Disable SCP Mobile log	43
Figure 8-4.	ADB logcat output	43
Figure 8-5.	Performance Budget log example	53
Figure 8-6.	Flight record console mode example	55

Figure 8-7. Button instruction	56
Figure 8-8. Flight record GUI mode interface	57
Figure 8-9. Detailed information	57
Figure 8-10. Task wake up event	58
Figure 8-11. Example of Queue event	58
Figure 8-12. Detailed information of queue event	58
Figure 8-13. Example of SW timer	58
Figure 8-13. Example of critical section	59
Figure 8-14. Example of ISR	59
Figure 8-15. Example of OS tick	59
Figure 9-1. SCP recovery behavior	69

List of Tables

Table 1-1. Hardware specification	6
Table 1-2. Architecture specification	9
Table 6-1. The standard DRAM access flow	25
Table 6-2. The default remapping rule table	29
Table 7-1. The EINT and GPIO mapping ball name	38
Table 7-2. The GPIO control register table	39
Table 8-1. PRINTF use scenario	41
Table 8-2. UART pin name	42
Table 8-3. Compiler options and definitions	52
Table 8-4. Compiler options	54
Table 8-5. Definitions of Flight Record	54
Table 9-1. Compiler options and definitions	60
Table 9-2. Exception type	60

1 SCP Overview

System Companion Processor (SCP) is a sub-system designed to perform always-on tasks even when the system is in low power state.

1.1 Hardware Architecture

SCP consists of dedicate processor(s), SRAM, DMA, and peripherals, e.g. ,I2C, GPIO. Since MT6853/6885, an internal DSP and a new architecture has been introduced, enabling always-on applications to run at lower power and achieve better performance.

- Processor: Embedded MDSP RV55NN x 1 and 2 hardware threads, each core with RISC-V architecture
 - Single precise floating point
 - Compressed instruction
 - DSP ISA for voice acceleration
 - Inside NN engine for vector instruction
 - Individual I cache 32K, D cache 32K for each core
 - OPPs from 266 to 800MHz
- Memory: 2MB TCM
- DMA throughput:
 - Data transfer from TCM to DRAM: 63.8 MB/s
 - Data transfer from DRAM to TCM: 39.9 MB/s
- Peripherals:
 - I2C x 1, I3C x 6
 - SPI x 4
 - UART x 2
- Details

Table 1-1. Hardware specification

		MT6897
Core		RV55NN x 1 (2 HW threads per core)
Cache	L1\$	I\$/ D\$: 32KB/32KB
	L2\$	256KB
TCM	L1TCM	NA
	L2TCM	2MB
Peripherals		• I2C x 1 • I3C x 6 • SPI x 4 • UART x 2
DMA		8 channels (0&1 reserved for i2c)
VoW I/F		3-mic
Operating Frequency		800M Hz@Vscp 0.75V
Performance (CoreMark@Vmin)		TBD
Power Efficiency (CoreMark/mW)		TBD

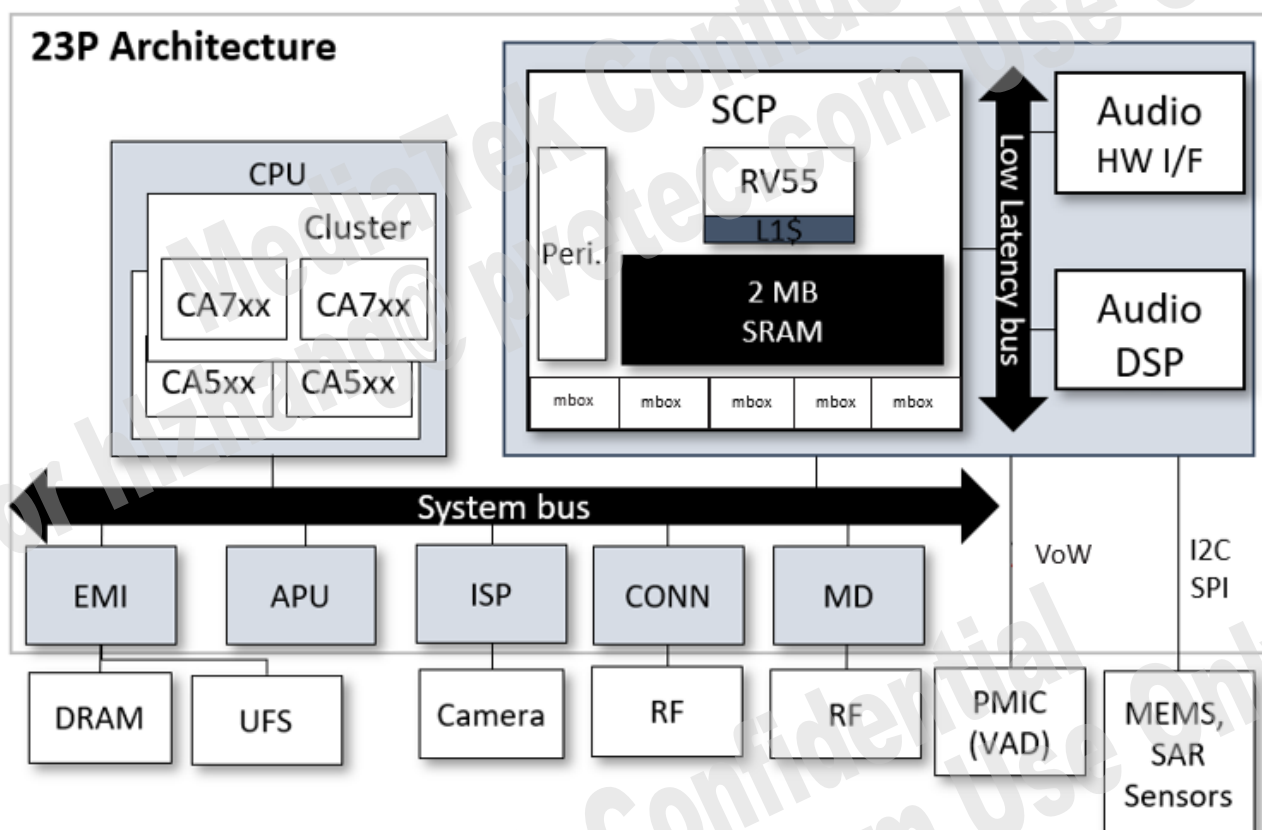


Figure 1-1. MTK 23P SCP architecture

1.2 Software Architecture

SCP SW is based on AWS FreeRTOS v10.1.010.1.0.1 which is a real time Kernel in MIT v2 license, supporting multi-task, mutex, semaphore and software timer.

The SW package also includes additional middleware such as audio and sensor hub which will not be detailed in this document.

SCP communicates with Linux Kernel via IPI (Inter Processor Interrupt) based on OpenAMP rpmsg/remoteproc framework. Similarly mechanisms are provided to enable co-work between AP and SCP:

- IPI – Chapter 0
- HW semaphore – Chapter 0
- Logger – Chapter 8.2
- Recovery – Chapter 0

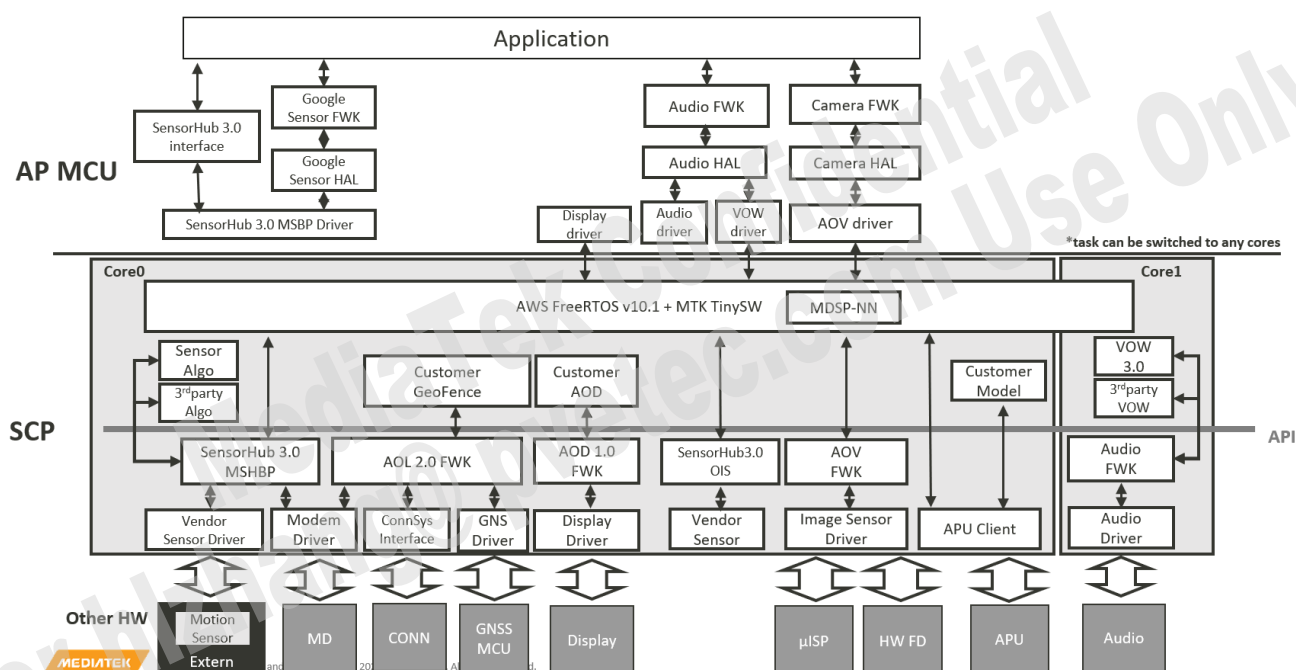


Figure 1-2. MTK MT6897 SCP software architecture

1.3 Summary

Table 1-2. Architecture specification

Item	Value
Platform	MT6897
Project	\$PROJECT
Linux version	6.1
FreeRTOS version	10.1.0.1
ISA	RV55-NN
L2 TCM	2MB

2 Source Tree

SCP source tree includes bootloader (e.g., LK), Linux Kernel and FreeRTOS, Listed as below:

2.1 Little Kernel Bootloader

```
vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/common
defined./scp
```

2.2 Linux Kernel Driver

- SCP driver path
kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp
- SCP DTS path
kernel/kernel_device_modules-6.1/arch/arm64/boot/dts/mediatek/mt6897.dts

2.3 FreeRTOS Tree

- RTOS Kernel
vendor/mediatek/proprietary/tinysys/kernel/FreeRTOS_v10.1.0.1
- Platform and peripheral drivers
vendor/mediatek/proprietary/tinysys/scp
vendor/mediatek/proprietary/tinysys/common
- Libraries
vendor/mediatek/proprietary/tinysys/scp/middleware
- Toolchain
prebuilts/clang/md32rv/linux-x86/CodeLine_212

3 Build System

Before programming, developers might want to know how to compile with basic SCP configuration. This chapter will show where/how configurations are arranged, and how to produce SCP image by commands.

3.1 Configuration Files

3.1.1 Little Kernel Bootloader

vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/mt6897/rules.mk

- `MODULES_DEP += platform/$(PLATFORM)/common/scp/RV`

3.1.2 Linux Kernel

Kernel/kernel_device_modules-6.1/arch/arm64/configs/mgk_64_k61_defconfig

- Enable/disable SCP driver: `CONFIG_MTK_TINYSYS_SCP_SUPPORT`
- Features switch, such as Voice Wakeup and Sensor Hub

3.1.3 FreeRTOS

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/platform.mk

- Default configurations of the platform
- Extra CFLAGS
- Extra LDFLAGS
- Driver/middleware C objects and include path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/project.mk

- Project-specific configuration
- Customize project by overriding options in `platform.mk`

3.2 Build Commands

There are several ways to build SCP images. Two of them are listed in ALPS SW packages.

3.2.1 Standalone

This method can build SCP firmware quickly, especially useful during development.

- The output path: `./tinysys_out`
- SYNOPSIS: `PROJECT=XXX TARGET_BOARD_PLATFORM=XXX$PROJECT BUILD_TYPE=[release, debug] make`
- e.g.,

```
$ cd vendor/mediatek/proprietary/tinysys/scp
$ PROJECT=k6897v1_64 TARGET_BOARD_PLATFORM=mt6897 BUILD_TYPE=debug make -j24
```

3.2.2 With AOSP Hierarchy

- The output path: `out/target/product/$PROJECT/obj/TINYSYS_OBJ/tinysys-scp_intermediates/RV55_A/scp`
- e.g.,:

```
$ make tinysys-scp -j24
$ mmm vendor/mediatek/proprietary/tinysys/scp -j24
$ cd vendor/mediatek/proprietary/tinysys/scp && mm -j24
$ mmm vendor/mediatek/proprietary/tinysys/scp:tinysys-scp -j24
```

3.3 Image Layout

- Partition in EMMC/UFS: scp1/scp2
 - scp1: Main and active partition
 - scp2: Backup for AB system
- Image: scp.img
 - Consists of:
 - tinysys-scp-RV55_A.bin: firmware/data located in SRAM
 - tinysys-scp-RV55_A.elf: elf with symbol, for debug purpose
 - tinysys-scp-RV55_A_DRAM.bin: firmware/data located in DRAM

4 Boot Sequence

After the SCP image is ready, you need to know how the image are loaded into SRAM/DRAM and how SCP is initiated to run. The whole flow is completed by LK bootloader/Linux kernel/SCP firmware together, as below:

- LK Bootloader
 - (vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/common/scp/scp.c)
 - Allocate permanent DRAM memory for SCP image
 - Load/verify SCP image
 - Setting EMI MPU (AP read-only)
- Kernel
 - Initial setting (mbox/ipi/logger/...)
 - Kick SCP
- SCP
 - Loader (vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/MT6897/platform/boot55.S)
 - Load SCP image to SRAM
 - Jump to FreeRTOS
 - FreeRTOS
 - (vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/MT6897/platform/src/main.c)
 - Driver init
 - Setup MPU

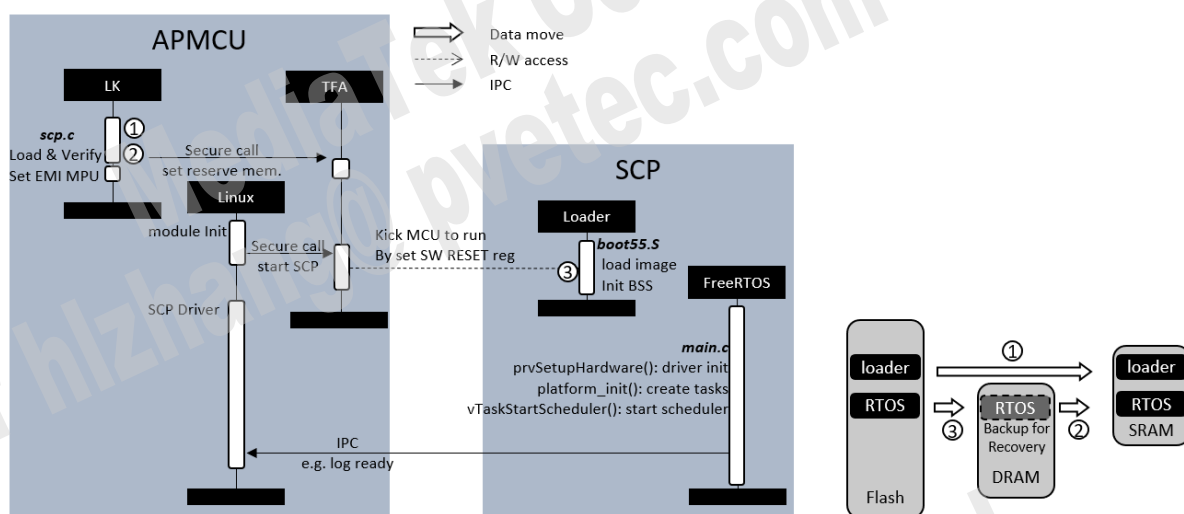


Figure 4-1. SCP boot up flow overview

5 Inter Processor Interrupt (IPI)

IPI is a mechanism to pass messages between drivers in Linux and FreeRTOS. It consists of:

- A piece share memory: To exchange data
- A set of interrupts: To notify each other

Figure 5-1 shows the Tinsys IPI architecture. The SW architecture consists of four layers

1. Synchronization layer: Public API for communication between AP and Tinsys
2. Rpmmsg layer: Blocking/non-blocking send functions
3. Queue layer: Queue operation and management functions
4. Physical layer: Physical hardware operations

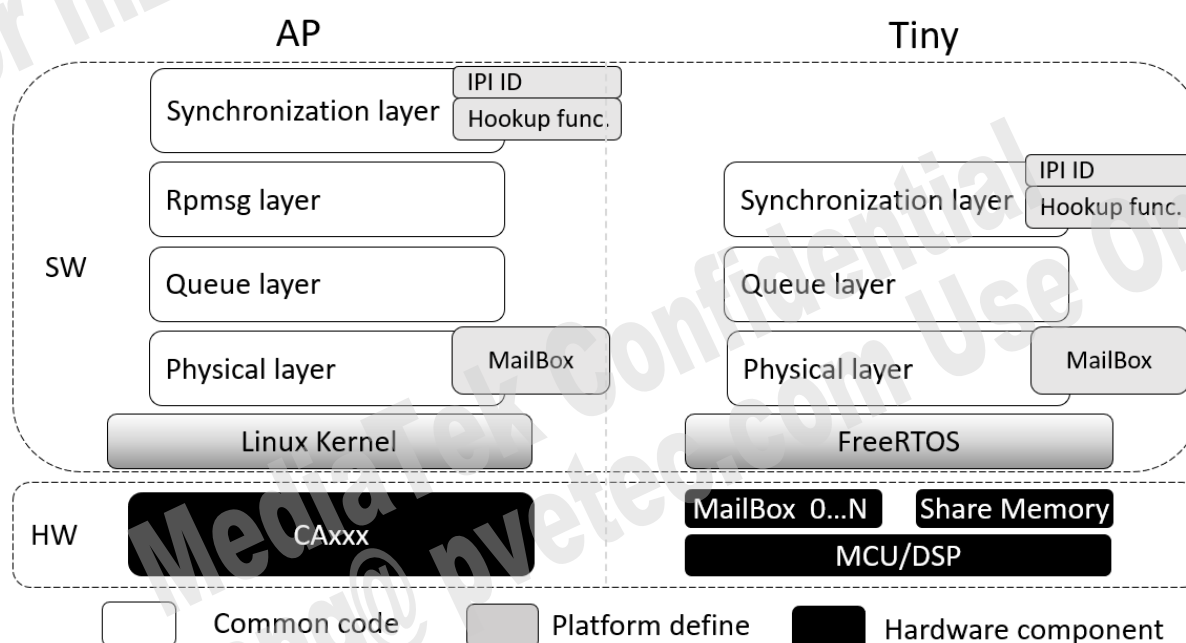


Figure 5-1. Tinsys common IPI architecture

The MailBox are only predefined. Developers use the flowing API to send IPI to Tinsys, including IPI ID and register the call back function (ipi_cb).

5.1 Usage on the SCP Side

- i. Add a new IPI ID information in `ipi_id.h`, `ipi_table.h`, and `mbox_pin.h`. Sending and receiving information are described by different IPI tables.

- File Description

- `ipi_id.h`: The definition IPI ID (must unique)
- `ipi_table.h`: The definition of IPI pin table (The sum of pin slots cannot exceed mbox slots)
- `mbox_ipi.h`: The definition of IPI pin slot

- Path

vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt6897/mbox/ipi_id.h

vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt6897/mbox/ipi_table.h

vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt6897/mbox/mbox_pin.h

- IPI_TABLE (Sender Pin Table Description)

```
/* mbox pin structure, this is for send definition,
 * ipi=endpoint=pin
 * mbox :          (mbox number)mbox number of the pin, up to 16
 * offset :        (slot)msg offset in share memory, 4 bytes alignment, up to
1024*4 KB
 * send_opt :      (opt)send opt, 0:send ,1: send for response
 * lock_opt :      (opt)option 0: mutex, 1: busy wait
 * msg_size :      (slot)message size in words, 4 bytes alignment
 * pin_index :     (bit offset)pin index in the mbox
 * ipi_id :        (ipi_id)ipi_id in the mbox
 * mutex :         (mutex)mutex for remote response
 * sema_ack :      (sema_ack)completion for remote response
 * send_record :   send pin record information
 */
struct pin_send {
    unsigned int mbox:4,
    offset:20,
    send_opt:2,
    lock_opt:2;
    unsigned int msg_size;
    unsigned int pin_index;
    unsigned int ipi_id;
    SemaphoreHandle_t mutex;
    SemaphoreHandle_t sema_ack;
    struct mtk_mbox_send_record send_record;
};
```

- IPI_TABLE(Receiver Pin Table Description)

```

/*
 * mbox pin structure, this is for receive definition,
 * ipi=endpoint=pin
 * mbox:                (mbox number)mbox number of the pin, up to 16
 * offset:              (slot)msg offset in share memory, 4 bytes alignment,
up to 1024*4 KB
 * recv_opt:            (opt)recv option, 0:receive ,1: response
 * lock_opt:            (opt)option 0: mutex, 1: busy wait
 * buf_full_opt:        (opt)buffer option 0:drop, 1:assert, 2:overwrite
 * cb_ctx_opt:          (opt)callback option 0:isr context, 1:process context
 * msg_size:            (slot)msg used slots in the mbox, 4 bytes alignment
 * pin_index:           (bit offset)pin index in the mbox
 * ipi_id:              (ipi_id)ipi_id in the mbox
 * notify:              (completion)notify process
 * mbox_pin_cb:         (cb)cb function
 * pin_buf:             (void*)buffer point
 * prdata:              (void*)private data
 * recv_record:         receive pin record information
 */
struct pin_recv {
    unsigned int mbox:4,
                offset:20,
                recv_opt:2,
                lock_opt:2,
                buf_full_opt:2,
                cb_ctx_opt:2;

    unsigned int msg_size;
    unsigned int pin_index;
    unsigned int ipi_id;
    SemaphoreHandle_t notify;
    void (*mbox_pin_cb) (unsigned int id, void *prdata, void *data, unsigned int
    len);
    void *pin_buf;
    void *prdata;
    struct mtk_mbox_recv_record recv_record;
};

```

- Example of adding new IPI ID

```
##Add IPI_NEW_ID at ipi_id.h
enum {
    .
    .
    IPI_NEW_ID =39,
    IPI_COUNT
};
```

- Example of adding new IPI message size

```
##Add PIN_IPI_NEW_ID_SIZE at mbox_pin.h

#define PIN_IPI_NEW_ID_SIZE 2
```

- Example of adding new IPI pin

The following guidelines cannot be violated.

The total msg_size of send_table and rcv table in the same mbox must not exceed 64.

The new ping setting must be buddled with the same mbox number

```
##Add new pin setting for send or rcv table at ipi_table.h

Struct pin_send mbox_pin_send_table [] = {
    {0, 0, 0, 0, ..., {}},
    .
    .
    {0, 0, 0, 0, PIN_IPI_NEW_ID_SIZE, 0, IPI_NEW_ID, 0, 0, {}},
    {1, 0, 0, 0, ..., {}},
    .
    .
    {2, 0, 0, 0, ..., {}},
    .
    .
}
```

ii. Register an IPI handler

```
int ipi_register(unsigned int ipi_id, void *cb, void *prdata, void *msg)
```

Description

To register IPI handler

Parameters

id: id declared in ipi_id.h

cb: IPI handler, a callback

prdata: IPI handler parameter, for customized

msg: msg buffer, data recv from AP

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

BUSY: the channel is busy. Need to retry.

• Example

```
#include "scp_ipi.h"
```

```
int ret;
```

```
ret = ipi_register(IPI_NEW_ID, ipi_cb, 0, "ipi_cb name");
```

```
if (ret != IPI_ACTION_DONE)
```

```
    PRINTF_E("Register IPI failed\n");
```

```
void ipi_cb(int id, void *prdata, void *data)
```

```
{
```

```
    /* data: received message from kernel */
```

```
    unsigned int rcv_data = *(unsigned int *)data;
```

```
    ...
```

```
}
```

iii. Receive an IPI

```
int ipi_recv(unsigned int ipi_id)
```

Description

API for apps to receive an IPI from AP

Parameters

id: IPI id declared in ipi_id.h

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

- Example:

```
#include "scp_ipi.h"

static void xxx_ipi_task(void *pvParameters)
{
    int ret;
    while(1) {
        ret = ipi_recv(IPI_NEW_ID);
        If (ret != DONE)
            PRINTF_E("Send IPI failed\n");
    }
}
```

```
int ipi_recv_reply(unsigned int ipi_id, void *reply_data, int len)
```

Description

API for apps to receive an IPI from AP and reply

Parameters

id: IPI id declared in ipi_id.h

reply_data: the message which will be replied to Linux kernel

len: message length(4 bytes as a unit)

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

- Example:

```
#include "scp_ipi.h"

static void new_ipi_task(void *pvParameters)
{
    int ret;
    while(1) {
        ret = ipi_recv_reply(IPI_NEW_ID, (void *)&reply_data, reply_data_len);
        If (ret != DONE)
            PRINTF_E("Send IPI failed\n");
    }
}
```

iv. Send an IPI

```
int ipi_send(unsigned int ipi_id, void *data, int len, unsigned long retry_timeout)
```

Description

API for apps to send an IPI to AP

Parameters

id: IPI id declared in ipi_id.h

data: the message which will be sent to Linux kernel

len: message length(4 bytes as a unit)

retry_timeout: the times of retry

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

• Example

```
#include "scp_ipi.h"
```

```
ipi_status ret;
```

```
ret = ipi_send (IPI_NEW_ID, (void *)&msg, msg_size, 0, IPI_SCP2AP);
```

```
if (ret != DONE)
```

```
    PRINTF_E("Send IPI failed\n");
```

v. IPI_STATUS

- If any problem occurs in run-time using, please check API return value first, the reference of return value is shown below.

#define IPI_ACTION_DONE	0	
#define IPI_ILLEGAL	-1	/* illegal ipi index */
#define IPI_DUPLEX	-2	/* the ipi has be registered */
#define IPI_UNAVAILABLE	-3	/* can't find this ipi pin define */
#define IPI_NO_MSGBUF	-4	/* ipi receiver doesn't has message buffer */
#define IPI_NO_MEMORY	-5	/* the message length is large than defined */
#define IPI_PIN_BUSY	-6	/* send message timeout */
#define IPI_RECV_TIMEOUT	-7	/* receive message timeout */
#define IPI_MBOX_ERR	-99	/* some error from physical layer */

5.2 Usage on the Kernel Side

i. Add a new IPI ID and pin table in `scp.h` and `mt6897.dts`

- File Description

- `scp.h`: The definition of IPI (ID must sync with SCP side)
- `mt6897.dts`: The definition of pin table (pin table must sync with SCP side)

- Path

kernel-**Error! Reference source not found.**`/drivers/misc/mediatek/scp/include/scp.h`

kernel-**Error! Reference source not found.**`/arch/arm64/boot/dts/mediatek/mt6897.dts`

- SCP IPI Description in DTS

- Example for adding new IPI ID and message size

```
## Add new IPI ID and message size at scp.h
```

```
.
.
.
#define PIN_IPI_NEW_ID_SIZE      2
.
.
.
enum {
    .
    .
    IPI_NEW_ID = 39,
    SCP_IPI_COUNT
}
```

- Example of adding new IPI pin setting

```
scp: scp@1cb00000 {
    .
    .
    send_table =
        // <id, mbox, send_size>
    .
    .
    <...>;
    recv_table =
        //< id, mbox, recv_size, recv_opt>,
        < 39, 0, 2, 0>,
    .
    .
    <...>;
}
```

ii. Register an IPI

```
int mtk_ipi_register(struct mtk_ipi_device *ipidev, int ipi_id, mbox_pin_cb_t cb,
void *prdata, void *msg)
```

Description

To register IPI handler

Parameters

ipidev: ipidev declared in scp.h

ipi_id: id declared in scp.h

cb: IPI handler, a callback

prdata: IPI handler parameter, for customized

msg: msg buffer, data recv from SCP

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

iii. Send an IPI

```
int mtk_ipi_send(struct mtk_ipi_device *ipidev, int ipi_id, int opt, void *data,
int len, int timeout)
```

Description

API for apps to send an IPI to scp

Parameters

ipidev: ipidev declared in scp.h

ipi_id: IPI id declared in scp.h

opt: IPI_mode, IPI_SEND_WAIT or IPI_SEND_POLLING, should be sync with pin table

data: the message which will be sent to SCP

len: message size(4 bytes as a unit)

timeout: busy wait , as ms per unit

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

SCP_NOT_READY: scp is not ready. Need to retry.

iv. Receive an IPI

```
int mtk_ipi_recv(struct mtk_ipi_device *ipidev, int ipi_id)
```

Description

API for apps to receive an IPI from scp

Parameters

ipidev: ipidev declared in scp.h

ipi_id: IPI id declared in scp.h

Return values

DONE: complete successfully

ERROR: something wrong in lower layer driver, i.e. mbox

SCP_NOT_READY: scp is not ready. Need to retry.

6 DRAM Access Procedure

Because DRAM and the system bus could be turned off regardless of SCP’s status, SCP developers **MUST** follow the following procedure to avoid system hang. Overview of sharing the information by DRAM between AP and SCP, is shown below.

- Note: Please refer to [Section 6.3](#) to get reserved DRAM address on SCP side.

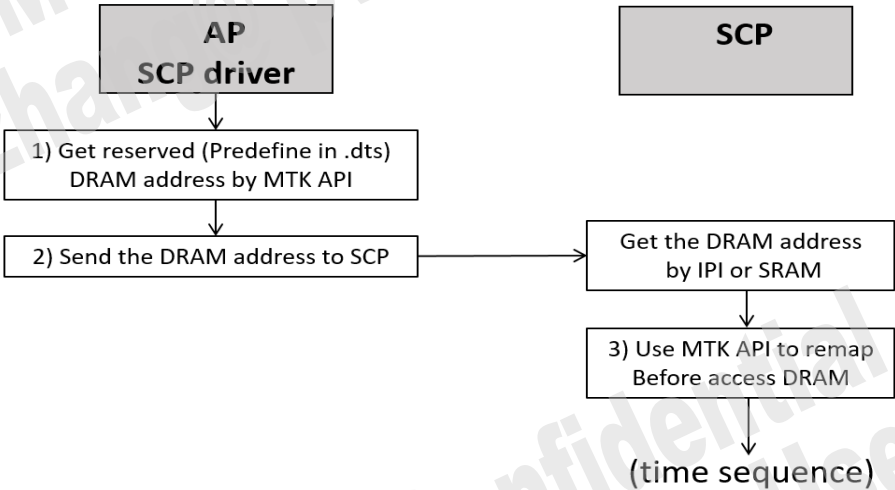


Table 6-1. The standard DRAM access flow

#	Steps	Note
0	Check/add predefined table in scp_reserve_mblock[]	
1	SCP Linux driver gets reserved physical address by APIs	phys_addr_t scp_get_reserve_mem_phys (scp_reserve_mem_id_t id) phys_addr_t scp_get_reserve_mem_virt (scp_reserve_mem_id_t id) phys_addr_t scp_get_reserve_mem_size (scp_reserve_mem_id_t id)
2	SCP Linux driver sends the address to SCP via IPI	scp_ipi_send (uint32 id, void* buf, uint len)
3	SCP accesses DRAM via API	uint32_t ap_to_scp (uint32_t ap_addr)
4	Enable DRAM before using	void dvfs_enable_DRAM_resource(scp_reserve_mem_id_t dma_id) void dvfs_disable_DRAM_resource(scp_reserve_mem_id_t dma_id)

6.1 Reserving Memory in Linux Kernel

To reserve a space in DRAM to exchange data between AP and SCP, entries in `scp_reserve_mblock[]` should be added first. The definition can be found here:

- Path

```
{lk2_repo}/platform/mediatek/mt6897/rules.mk
{kernel_repo}/arch/arm64/boot/dts/mediatek/mt6897.dts
{kernel_repo}/drivers/misc/mediatek/scp/include/scp.h
{kernel_repo}/drivers/misc/mediatek/scp/rv/scp_reservedmem_define.h
```

6.1.1 Register an User in Shared Memory

step1. Add new ID in `scp_reserve_mem_id_t`

```
{kernel_repo}/drivers/misc/mediatek/scp/include/scp.h
enum scp_reserve_mem_id_t {
    SCP_A_SECDUMP_MEM_ID = 0,
    VOW_MEM_ID,
    SENS_MEM_ID,
    ...
    USER_MEM_ID, // add here
    NUMS_MEM_ID,
}
```

step2. Add member to data structure in `scp_reserve_mblock`

```
{kernel_repo}/drivers/misc/mediatek/scp/rv/scp_reservedmem_define.h
static struct scp_reserve_mblock scp_reserve_mblock[] = {
{
    .num = SCP_A_SECDUMP_MEM_ID,
    .start_phys = 0,
    .start_virt = 0,
    .size = 0,
},
{
    .num = VOW_MEM_ID,
    .start_phys = 0x0,
    .start_virt = 0x0,
    .size = 0x0,
},
...
{
    .num = USER_MEM_ID,
    .start_phys = 0x0,
    .start_virt = 0x0,
    .size = 0x0,
},
}
```

step3. Add dts node of reserved memory

```
{kernel_repo}/arch/arm64/boot/dts/mediatek/mt6897.dts
scp: scp@1cb00000{
...
}
```

```

scp_mem_tbl = <0 0x0>,
<1 0xca700>, /* vow */
<2 0x100000>, /* sensor main*/
...,
id, size>;      /*user_mem*/
};

```

step4. Add memory size increment of SCP shared memory in makefile

```

{lk2_repo}/platform/mediatek/mt6897/rules.mk
SCP_RESERVED_SHARE_DRAM_SIZE:= 0x          //add increment to this variable

```

6.1.2 Extend Shared Memory Size

Take sensor for example, add 1.5MB

```

{kernel_repo}/arch/arm64/boot/dts/mediatek/mt6897.dts
scp: scp@1cb00000{
...
scp_mem_tbl = <0 0x0>,
<1 0xca700>, /* vow */
<2 0x280000>, /* sensor main original value is 0x100000 */
...,
};

```

```

{lk2_repo}/platform/mediatek/mt6897/rules.mk
SCP_RESERVED_SHARE_DRAM_SIZE:= 0x680000    // original = 0x500000

```

6.2 Get Reserved Memory by ID

Because the AP CPU uses virtual address and SCP uses physical address, both the two types of address are supplied. The following APIs are used to get virtual/physical address and size by given IDs which are declared in [Section 6.1](#).

- API

```
phys_addr_t scp_get_reserve_mem_phys(scp_reserve_mem_id_t id)
phys_addr_t scp_get_reserve_mem_virt(scp_reserve_mem_id_t id)
phys_addr_t scp_get_reserve_mem_size(scp_reserve_mem_id_t id)
```

- Header Path

kernel-**Error! Reference source not found.**/drivers/misc/mediatek/scp/rv**Error! Bookmark not defined.**/scp_helper.h

- Return Value

- The start address of the reserved memory, or
- 0x0: Means no mapping

After getting the physical address, developers **MUST** pass it to SCP via IPI, as mentioned in [Section 0](#).

6.3 Get Reserved DRAM Region on SCP Side

To keep security state and avoid malicious IPI messages, SCP_DRAM_REGION is introduced to provide the information of reserved DRAM region on SCP directly. The following APIs are used to get physical address and size by given IDs which are declared in [Section 6.1](#).

- API

```
bool scp_get_reserve_mem_by_id(uint32_t id, void **ap_addr, size_t *size)
```

- Header Path

vendor/mediatek/proprietary/tinysys/scp/drivers/common/dram_region_mgmt/scp_dram_region.h

- Return Value

- True: the value of *ap_addr and size are updated to start address and size of given ID.
- False: the *ap_addr and size are not updated.

For the codes which may be common part on different projects, CFG_SCP_DRAM_REGION_MANAGE can help to keep compatibility. For example,

```
#ifdef CFG_SCP_DRAM_REGION_MANAGE
#include "scp_dram_region.h"
#define USER_MEM_ID 11 //id number from dts
#endif

void init_memory(...) {
...
#ifdef CFG_SCP_DRAM_REGION_MANAGE
void *addr;
size_t size;

if (scp_get_reserve_mem_by_id(USER_MEM_ID, &addr, &size))
    set_dram_addr(addr, size);
else
...
}
```

```
#else
// msg.addr and msg.size from ipi message
set_dram_addr(msg.addr, msg.size);
#endif
...
}
```

6.4 Remap DRAM Address from AP View to SCP View

SCP is a 32-bit system and can only access maximum 4G (0xffffffff) address. If SCP applications need to access higher than 4G, the following APIs should be used to remap address to SCP memory window. Table 6-2 shows the relation between AP and SCP.

Table 6-2. The default remapping rule table

Name	SCP/DMA		Size	AP side	
External Memory	0x1000_0000	0x1FFF_FFFF	256MB	0x6000_0000	0x6FFF_FFFF
External Memory	0x2000_0000	0x2FFF_FFFF	256MB	0x7000_0000	0x7FFF_FFFF
External Memory	0x5000_0000	0x5FFF_FFFF	256MB	0x0000_0000	0x0FFF_FFFF
External Memory	0x6000_0000	0x6FFF_FFFF	256MB	0x1000_0000	0x1FFF_FFFF
External Memory	0x9000_0000	0x9FFF_FFFF	256MB	0x8000_0000	0x8FFF_FFFF
External Memory	0xA000_0000	0xAFFF_FFFF	256MB	0x9000_0000	0x9FFF_FFFF
External Memory	0xD000_0000	0xDFFF_FFFF	256MB	0x2000_0000	0x2FFF_FFFF
External Memory	0xE000_0000	0xEFFF_FFFF	256MB	0x3000_0000	0x3FFF_FFFF
External Memory	0xF000_0000	0xFFFF_FFFF	256MB	0x5000_0000	0x5FFF_FFFF

- API
 - uint32_t ap_to_scp(uint32_t ap_addr);
 - uint32_t scp_to_ap(uint32_t scp_addr);
- Header Path
vendor/mediatek/proprietary/tinysys/scp/drivers/common/dma/inc/dma_api.h
- Return Value
 - The mapped address, or
 - 0x0: Means no mapping

6.5 Request System Bus and DRAM

Because the system bus and DRAM will enter sleep mode when no data is being transmitted or system suspends, the following APIs **MUST** be invoked to make sure DRAM is ready to access.

- API for task context
 - void dvfs_enable_DRAM_resource(scp_reserve_mem_id_t dma_id): before DRAM access
 - void dvfs_disable_DRAM_resource(scp_reserve_mem_id_t dma_id): after DRAM access
- Note: It will take 5ms to wake up DRAM when 26M clock is gated.

- API for ISR context

- void dvfs_enable_DRAM_resource_from_isr(scp_reserve_mem_id_t dma_id)
- void dvfs_disable_DRAM_resource_from_isr(scp_reserve_mem_id_t dma_id)

- Header Path

vendor/mediatek/proprietary/tinysys/common/drivers/dma/v3/inc/dma.h

vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/MT6897/dvfs/inc/dvfs.h

7 Drivers Guide

7.1 Driver Initiation

Due to that the multi-thread is not enabled when driver is initiated, developers **MUST**

- Put drivers init function in platform_init().
- Never use block functions in driver init function because it will be blocked forever. For example:
 - vTaskDelay
 - HW semaphore
 - Busy loop, e.g., polling registers

7.2 Add a New Driver

Here are steps to add a new driver to the source tree:

1. Put the driver body in the appropriate folder (choose one of following folders)

- Path

```
vendor/mediatek/proprietary/tinysys/common/* common drivers for tinysys */
vendor/mediatek/proprietary/tinysys/scp/drivers/common/* common drivers for scp */
vendor/mediatek/proprietary/tinysys/scp/drivers/RV55_A/mt6897/drivers/* platform drivers */
```

2. Add a new compiler option

- Path

```
vendor/mediatek/proprietary/tinysys/scp/project/mt6897/platform/platform.mk
```

- Example: DMA driver

```
CFG_DMA_SUPPORT = yes
...
ifeq ($(CFG_DMA_SUPPORT), yes)
    INCLUDES += $(COMMON_DIR)/drivers/dma/v3/inc
    INCLUDES += $(SCP_DRIVERS_DIR)/common/dma/inc
    INCLUDES += $(DRIVERS_PLATFORM_DIR)/dma
    C_FILES += $(COMMON_DIR)/drivers/dma/v3/dma.c
    C_FILES += $(SCP_DRIVERS_DIR)/common/dma/dma_api.c
endif
```

7.3 Interrupt

MT6897 SCP supports 15 priority levels of interrupt, and the lower-level number with the higher priority. If more than one IRQ happen at the same time, CPU will serve the one with highest priority first. Never set interrupt priorities higher than 2. Level 0 is for the “watch dog” or “system fail”, level 1 is for “sleep control” interrupt. This chapter describes how to use interrupt in SCP.

- Path
vendor/mediatek/proprietary/tinysys/common/drivers/irq/v3/inc/irq.h

7.3.1 IRQ Registration

Before an IRQ is served, the driver developer must make an association between the IRQ ID and the corresponding handler. The IRQ is defined according to the structure of INTC_IRQ, and the driver developer following API to register handlers with IRQ IDs.

• Structure

```
struct INTC_IRQ
{
    uint8_t id;
    uint8_t group;
    uint8_t pol;
}
```

id: the irq number
group: from INTC_GRP_0 (the highest priority) to INTC_GRP_14 (the lowest priority)
pol: the polarity with INTC_POL_HIGH or INTC_POL_LOW

• API

```
int into_irq_request(struct INTC_IRQ *irq, irq_handler_t handler, void *userdata)
```

Description
Request an irq and register the handler, and default enable the irq handler.

Parameters
irq: the irq structure which is declared at intc.h
handler: irq handler
userdata: it will deliver to irq handler as a parameter

Return values
0: success
-1: request fail

• Example

```
struct INTC_IRQ INTC_IRQ_SYSTICK = {0, INTC_GRP_8, INTC_POL_HIGH};

#include "irq.h"

int ret;
ret = into_irq_request(&INTC_IRQ_SYSTICK, test_ist, NULL);
if (ret != 0)
    PRINTF_E("Register irq failed\n");
```

7.3.2 Enable IRQ

After IRQ handler is registered, SCP is ready to service. The next step is to enable a specific IRQ with the following API.

- API

```
int intc_irq_enable(struct INTC_IRQ *irq)
Description
    Enable specified irq served.
Parameters
    irq: the irq structure which is declared at intc.h
Return values
    0: success
    -1: request fail
```

- Example:

```
#include "irq.h"

int ret;
ret = intc_irq_enable(&INTC_IRQ_SYSTICK);
if (ret != 0)
    PRINTF_E("enable irq failed\n");
```

7.3.3 Disable IRQ

Opposite, when IRQs are not used temporarily, driver developers have to call the following API to make SCP stop service the IRQs.

- API

```
int intc_irq_disable(struct INTC_IRQ *irq)
Description
    Disable specified irq served.
Parameters
    irq: the irq structure which is declared at intc.h
Return values
    0: success
    -1: request fail
```

- Example

```
#include "irq.h"

int ret;
ret = intc_irq_disable(&INTC_IRQ_SYSTICK);
if (ret != 0)
    PRINTF_E("disable irq failed\n");
```

7.3.4 Wakeup Source Registration

When SCP is in the sleep state, an IRQ will not be serviced unless it is set as wakeup source. The following API is used to register an IRQ as a wakeup source.

- API

```
int intc_irq_wakeup_set(struct INTC_IRQ *irq, unsigned int wake_src)
```

Description

Set wakeup source for specified irq.

Parameters

irq: the irq structure which is declared at intc.h

wake_src: 1 for wakeup source, 0 for non-wakeup source

Return values

0: success

-1: request fail

- Example

```
#include "irq.h"

int ret;
ret = intc_irq_wakeup_set(&INTC_IRQ_SYSTICK, 1);
if (ret != 0)
    PRINTF_E("irq wakeup source setup failed\n");
```

Note:

1. ISR execution time must be as SHORT as possible and stack depth as LESS as possible
2. Never use block APIs, such as
 - HW semaphore API
 - Wait for IPI
 - Any APIs with polling external devices
3. Never use FreeRTOS API in ISR without FromISR postfix
 - It is a rewritten version with block-free API and quick implementation
4. Must use `portYIELD_FROM_ISR()` if there is a high priority task waken in ISR

7.4 Locks

SCP provides a spin lock mechanism for dual-core synchronization. To use the lock, the following configuration item needs to be enabled.

set CFG_ATOMIC_PLAT_SUPPORT = yes

- Path
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/platform.mk

- API

```
spinlock_t SYNC_SECTION lock;

void spinlock_lock(spinlock_t * lock)
void spinlock_unlock(spinlock_t * lock)
description
spin lock for Sync dual core.
parameters
lock: variable defined by SYNC_SECTION
Return values
NA.
```

- Header Path
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/inc/mtk_atomic.h

Note:

1. Never keep spin lock more than 1ms due to preemption is disabled when spin is locked and unlocked. For scenarios that do not span both cores, spin locks are not necessary.

7.5 DMA

Direct Memory Access (DMA) is a hardware feature that allows data to be copied between specified source/destination without involving the CPU. SCP DMA supports the following functions:

1. Burst AXI mode to accelerate memory transfers
2. 8 channels, meaning the engine can concurrently handle up to 8 transactions

- API

```
DMA_RESULT scp_dma_transaction(uint32_t dst_addr, uint32_t src_addr, uint32_t len, int8_t
scp_dma_id, int32_t ch)
DMA_RESULT scp_dma_transaction_dram(uint32_t dst_addr, uint32_t src_addr, uint32_t len,
int8_t scp_dma_id, int32_t ch)
description
Copy data from src_addr to dst_addr by specified DMA channel
parameters
dst_addr: destination address
src_addr: source address
len: length in bytes to copy
scp_dma_id: DMA ID
ch: channel ID
Return values
DMA_RESULT_DONE (=0) means success start
DMA_RESULT_NO_FREE_CH (=-1) means DMA hardware busy
```

- Header Path

```
vendor/mediatek/proprietary/tinysys/common/drivers/dma/v3/inc/dma.h
vendor/mediatek/proprietary/tinysys/scp/drivers/common/dma/dma_api.h
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/Error! Reference source not
found./mt_dma.h
```

- scp_dma_id

- It's the identity of DMA channel in mt_dma.h. It's recommended to use different dma_id with which debugging channel full issues will be easier.

- Limitations

- The maximum data size per transaction is 262140 bytes
- Use 4byte-aligned address to get the best performance

- Example:

```
ret = scp_dma_transaction(dst_buf + dst_w_pos, src_buf + src_r_pos, src_len,
LOGGER_DMA_ID, NO_RESERVED);
if (ret != DMA_RESULT_DONE) {
    PRINTF_E("log dma trans fail\u\n", ret);
    return 0;
}
```

7.6 Hardware Semaphore

Hardware semaphore is a special hardware which provides mutex-like flow control between Linux driver and FreeRTOS.

There are 16 sets in SCP.

The following APIs make Hardware semaphore easy to use. They work on both SCP and Linux driver. Just make sure that the right header is included and the flags are the same.

- API

```
int semaphore_get(unsigned int flags)
int semaphore_release(unsigned int flags);
description
    Semaphore between AP and SCP.
parameters
    flag: 0 ~15 for 16 sets in SCP
Return values
    0: get semaphore fail
    1: get semaphore success
```

- Header

Kernel/kernel_device_modules -6.1/drivers/misc/mediatek/scp/rv/scp_helper.h
vendor/mediatek/proprietary/tinysys/common/drivers/sem/v1/inc/sem.h

- Return

- 1: Succeed in getting semaphore
- 0: Fail to get semaphore

- Examples:

```
int get_semaphore;

while(1) {
    get_semaphore = semaphore_get(4)
    if (get_semaphore)
        break;
}
```

7.7 GPIO & EINT

SCP also provides GPIO an external interrupt so that external components such as gyro sensors can send events to SCP.

- GPIOs
 - Function set to: TP_GPIO?_AO, support TP_GPIO0_AO~TP_GPIO15_AO
- EINTs
 - Function set to Aux Func.0 (GPIO), support EINT0~15

Table 7-1. The EINT and GPIO mapping ball name

Ball Name	GPIO Reset Default Mode	EINT	Aux Func.0	Aux Func.4	Aux Func.6
EINT0	0	EINT0	B:GPIO0		B0:TP_GPIO0_AO
EINT1	0	EINT1	B:GPIO1		B0:TP_GPIO1_AO
EINT2	0	EINT2	B:GPIO2		B0:TP_GPIO2_AO
EINT3	0	EINT3	B:GPIO3		B0:TP_GPIO3_AO
EINT4	0	EINT4	B:GPIO4		B0:TP_GPIO4_AO
EINT5	0	EINT5	B:GPIO5		B0:TP_GPIO5_AO
EINT6	0	EINT6	B:GPIO6		B0:TP_GPIO6_AO
EINT7	0	EINT7	B:GPIO7		B0:TP_GPIO7_AO
EINT8	0	EINT8	B:GPIO8	B0:TP_GPIO8_AO	
EINT9	0	EINT9	B:GPIO9	B0:TP_GPIO9_AO	
EINT10	0	EINT10	B:GPIO10		B0:TP_GPIO10_AO
EINT11	0	EINT11	B:GPIO11		B0:TP_GPIO11_AO
EINT12	0	EINT12	B:GPIO12		B0:TP_GPIO12_AO
EINT13	0	EINT13	B:GPIO13		B0:TP_GPIO13_AO
EINT14	0	EINT14	B:GPIO14		B0:TP_GPIO14_AO
EINT15	0	EINT15	B:GPIO15		B0:TP_GPIO15_AO

7.7.1 GPIO Usage

The GPIO function must be set to TP_GPIO?_AO first. Please refer to GPIO pin mux setting document for details. GPIO control register table is shown in **Table 7-2**.

Table 7-2. The GPIO control register table

Register				
Offset	Name	Description	Access	Enumeration
0x25000	GPIO_DIR	GPIO direction	RW	0: Input 1: Output
0x25004	GPIO_OUT	GPIO output	RW	GPIO output [1:0]
0x25008	GPIO_IN	GPIO input	RO	GPIO input[1:0]
0x2500C	GPIO_PULL_EN	GPIO pull enable	RW	0: Disable 1: Enable
0x25010	GPIO_PULL_CTRL	GPIO pull control	RW	0: Pull down 1: Pull up

The control examples:

- Pull high GPIO 1
 - Set GPIO_DIR, 0x60525000[1] = 1
 - Set GPIO_OUT, 0x65025004[1] = 1
- Read GPIO 0
 - Set GPIO_DIR, 0x60525000[0] = 0
 - Read GPIO_IN, 0x60525008[0]
- Internal pull up GPIO 3
 - Set GPIO_PULL_EN, 0x6052500c[3] = 1
 - Set GPIO_PULL_CTRL, 0x60525010[3] = 1

7.7.2 EINT Usage

- Path

tinysys/common/drivers/eint/v02/src

- **API: Register EINT call back**

```
void mt_eint_registration(unsigned int eint_num, unsigned int sens, unsigned int pol,
                          void (EINT_FUNC_PTR) (int),
                          unsigned int unmask, unsigned int
is_auto_unmask)
description
    Register eint interrupt handler.
parameters
    eint_num: the EINT number to register
    sens: LEVEL_SENSITIVE, EDGE_SENSITIVE
    pol: HIGH_LEVEL_TRIGGER, LOW_LEVEL_TRIGGER
    EINT_FUNC_PTR: the ISR callback function
    Unmask: enable this EINT trigger after register
    Is_auto_unmask: auto reenable EINT trigger after finish a EINT service routine
Return values
    NA.
```

- Example

```
mt_eint_registration(eint_num, LEVEL_SENSITIVE, HIGH_LEVEL_TRIGGER, xxx_Isr,
EINT_INT_UNMASK, EINT_INT_AUTO_UNMASK_OFF);
```

- void mt_eint_dis_hw_debounce(unsigned int eint_num): Disable HW debounce
- void mt_eint_soft_set(unsigned int eint_num): Software trigger to clear specified EINT

8 Debug Methods

8.1 PRINTF_* Usage

Please **DO NOT** use printf in SCP SW, because it could be linked to C library and cause problem. Instead, PRINTF_* is used as shown in Table 8-1. Developers **MUST** include the header file <mt_printf.h> before using PRINTF_*. The log level and the use scenario of PRINTF_x is shown in Table 8-1.

Table 8-1. PRINTF use scenario

PRINTF_*	Level	Use Scenario	
PRINTF_E	<0>	Error conditions	
PRINTF_W	<1>	Warning conditions	User load
PRINTF_I	<2>	Informational	
PRINTF_D	<3>	Debug-level messages	Engineer load

8.2 Mobile Log

MTK Logger is an APK which records various logs into storage such as SD card. After launching it and enable SCP log, SCP log can be gotten in the following path:

Log path: /mobilelog/APLog_XXXX_XXXX_XXXXXX/scp_log_XXXX.curf

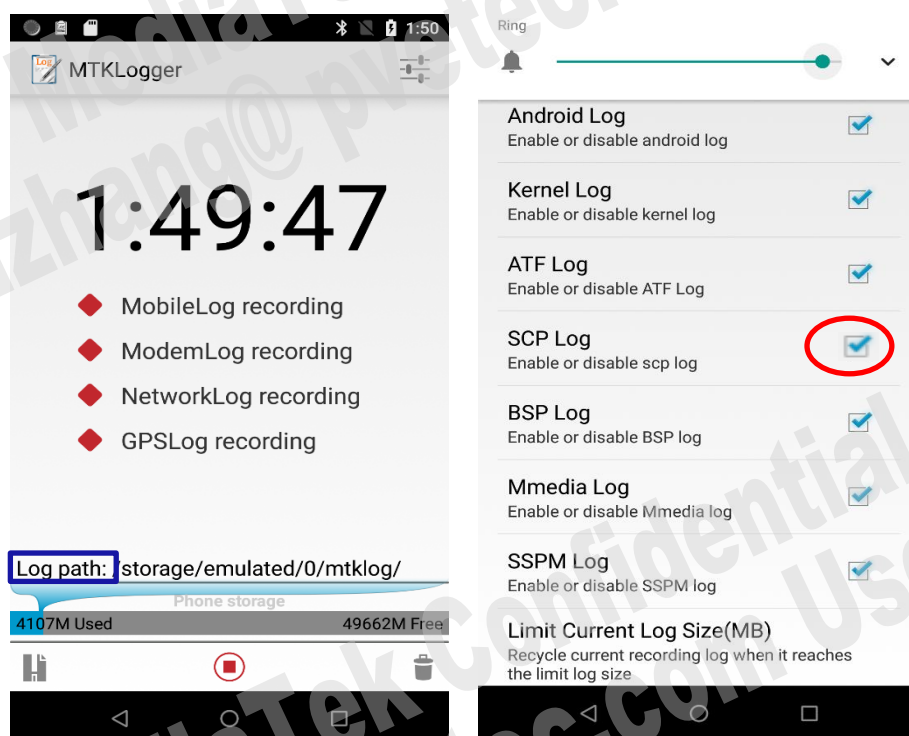


Figure 8-1. MTKLogger

8.3 UART

- Output pin

SCP has two dedicate UART. Please make sure the PC UART port connect to Pin ball name URXD1 and UTXD1 (as shown in Table 8-2) and then set the software compiler option as shown below.

Table 8-2. UART pin name

Pin name	Function
URXD1	SCP UART RX
UTXD1	SCP UART TX

- SW compiler option

- Path: Configure flags

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/platform.mk

- CFG_UART_SUPPORT = yes /* Uart enable, **Default No** */
- CFG_MTK_SCPUART_SUPPORT = yes /* Use SCP uart, **Default Yes** */

- UART terminal setting

- Baud rate: 921600

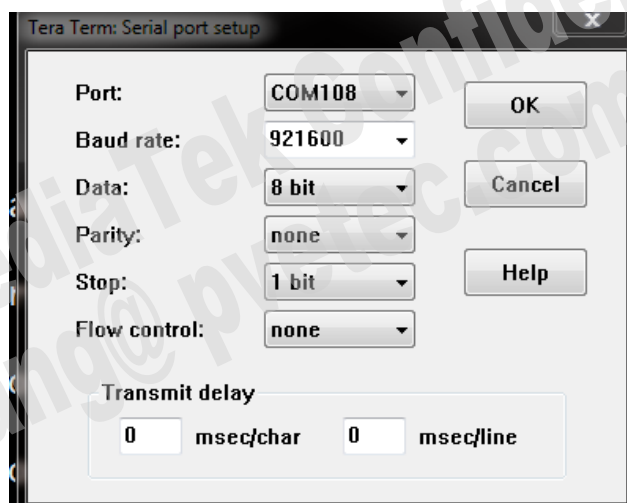


Figure 8-2. UART setting

8.4 ADB Logcat

ADB logcat is able to output SCP log directly from ADB or UART console.

- Usage:

- Make sure SCP Log in MTK Logger is disabled (as shown in Figure 8-3.)
- Enter shell and enter command `echo 1 > /sys/class/misc/scp/scp_mobile_log`
- Enter command `while true; do cat /dev/scp;done` and the log will output directly, as shown in Figure 8-4.

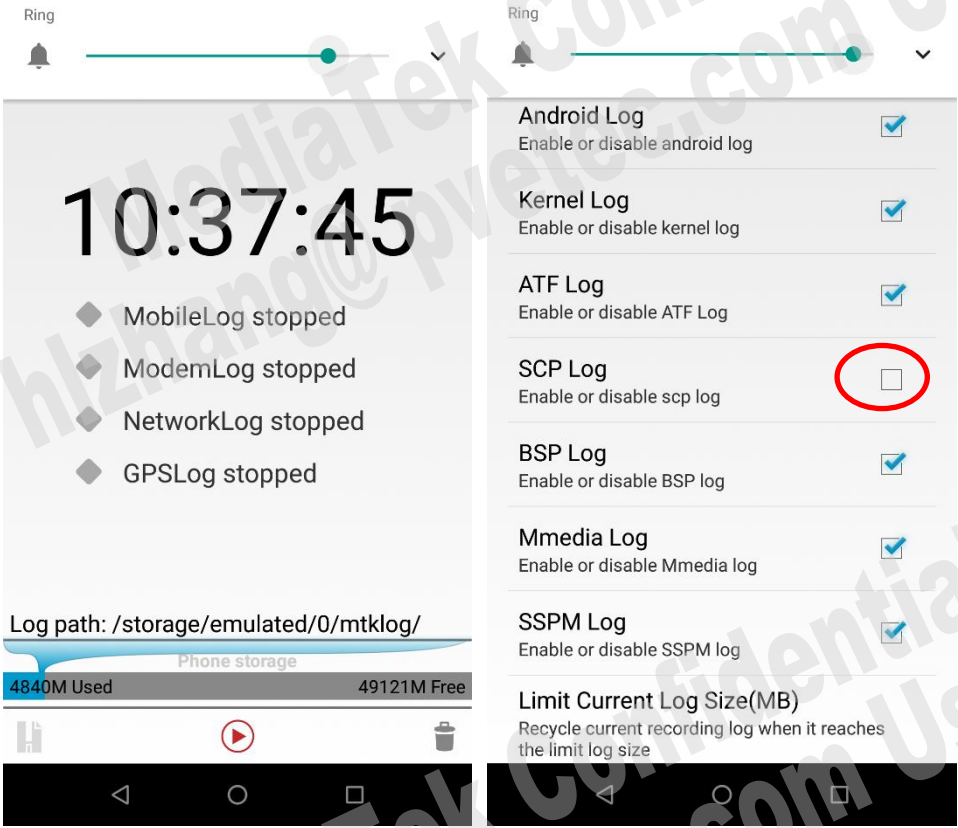


Figure 8-3. Disable SCP Mobile log

```
C:\Users\MTK11261>adb shell
k71v1_64_bsp:/ # echo 1 > /sys/class/misc/scp/scp_mobile_log
k71v1_64_bsp:/ # while true; do cat /dev/scp;done
103284, ap:39031836157017, ap_raw:39031836071402
raw_offset:2290053733, timestamp_offset_to_ap:2290053733
sync time counter_elapse:1126, ipi_transfer_time:86615
sync time scp:39039562168462, ap:39041852222325, ap_raw:39041852135710
raw_offset:2290053863, timestamp_offset_to_ap:2290053863
No sleep reasons: tmr=0, build=0, sema=0, lock=0, ipi=0, flag=4, slpbusy=0
sync time counter_elapse:1129, ipi_transfer_time:86846
sync time scp:39049578174332, ap:39051868228172, ap_raw:39051868141326
raw_offset:2290053840, timestamp_offset_to_ap:2290053840
sync time counter_elapse:1130, ipi_transfer_time:86923
sync time scp:39059594165433, ap:39061884219173, ap_raw:39061884132250
raw_offset:2290053740, timestamp_offset_to_ap:2290053740
sync time counter_elapse:1111, ipi_transfer_time:85461
sync time scp:39069610088456, ap:39071900142173, ap_raw:39071900056712
raw_offset:2290053717, timestamp_offset_to_ap:2290053717
No sleep reasons: tmr=0, build=0, sema=0, lock=0, ipi=0, flag=3, slpbusy=0
log en=1,update=1
sync time counter_elapse:1208, ipi_transfer_time:92923
sync time scp:39079626695403, ap:39081916749174, ap_raw:39081916656251
raw_offset:2290053771, timestamp_offset_to_ap:2290053771
^C
```

Figure 8-4. ADB logcat output

8.5 Exception Log Analysis

When exception happens, SCP will print exception log automatically. Developers can get it from UART or mobile log.

- **FAULT FETCH, FAULT LOAD, and FAULT STORE:** Due to the wrong access of a MPU protect address.
 - **FAULT FETCH:** PC jumps to wrong address
 - The system register: mepc will show the incorrect target address.
 - **FAULT LOAD, FAULT STORE:** Access a protected address
 - The system register mepc will show the wrong PC and mtval will show the incorrect address to be accessed.

```
[70.168] (0) exception: CAUSE_FAULT_LOAD
[70.168] (0)      exception pc: 0x00018b2c
[70.168] (0)      fault load address: 0xfffff1110
[70.168] (0) Regs dump
[70.168] (0) x0: 0xca0801e4 ra: 0x00018b24
[70.168] (0) sp: 0x00036ec0 gp: 0x00000000
...
[70.168] (0) t3: 0x00000000 t4: 0x00000011
[70.168] (0) t5: 0x00000000 t6: 0x00000000
[70.168] (0) pc: 0x00001454 mstatus: 0x05015800
[70.168] (0) mepc: 0x00018b2c
[70.168] (0) mcause: 0x00000005
[70.168] (0) mtval: 0xfffff1110
[70.168] (0) T Buffer (10)
[70.168] (0) 00 0x0000fdca:0x0002252c
[70.168] (0) 01 0x0002252c:0x0000fdce
...
[70.169] (0) 28 0x000108c0:0x0002368e
[70.169] (0) 29 0x00023698:0x000108c4
[70.169] (0) 30 0x000108ca:0x00024518
[70.169] (0) 31 0x00024524:0x00018b24
[70.169] (0) Code: 7545 85aa 0513 1105 <4108> 8593 1145 c188 9205
```

- **MISALIGNED FETCH, MISALIGNED LOAD, and MISALIGNED STORE:** Usually happen when the integer pointer (4 byte aligned) accesses a 1 or 2 bytes aligned address.
 - **MISALIGNED FETCH:** Jump to misaligned address
 - The system register: mepc will show the wrong address
 - **MISALIGNED LOAD, MISALIGNED STORE:** Access a misaligned address
 - The system register mepc will show the wrong PC and mtval will show the access address

- **ILLEGAL_INSTRUCTION:** The error caused by the inability to decode the instruction.
- **For example:**

```
[2.643] (0) exception: CAUSE_ILLEGAL_INSTRUCTION
[2.643] (0) Regs dump
[2.644] (0) x0: 0xa5a5a5a5 ra: 0xa5a5a5a5
[2.644] (0) sp: 0xa5a5a5a5 gp: 0xa5a5a5a5
...
[2.651] (0) t3: 0x00000000 t4: 0x00000000
[2.652] (0) t5: 0x00000000 t6: 0x00000000
[2.652] (0) pc: 0x00001454 mstatus: 0x05015880
[2.653] (0) mepc: 0x00201720
[2.654] (0) mcause: 0x00000005
[2.654] (0) mtval: 0x2ff13748
[2.655] (0) T Buffer (10)
[2.656] (0) 00 0x0000371a:0x00008fd6
[2.656] (0) 01 0x00008fdc:0x0000371e
...
[2.661] (0) 28 0x0000f2a4:0x00002eec
[2.662] (0) 29 0x00002eec:0x00006d52
[2.662] (0) 30 0x00006d5c:0x00002ef0
[2.663] (0) 31 0x0000f2a4:0x00003728
[2.664] (0) Code: d194 5133 4521 873f <3748> 2ff1 a5a5 a5a5 a5a5
```

8.5.1 Trace Buffer

Trace buffer log can help us to analyze the function call record that CPU have been executed. The log indicates the function call address and destination address. There are 32 entries, and could be gotten from the bottom of the exception log.

For example:

The address can be translated with tools like lldb or addr2line

```
[2.655] (0) T Buffer (10)
[2.656] (0) 00 0x0000371a:0x00008fd6
[2.656] (0) 01 0x00008fdc:0x0000371e
...
[2.661] (0) 28 0x0000f2a4:0x00002eec
[2.662] (0) 29 0x00002eec:0x00006d52
[2.662] (0) 30 0x00006d5c:0x00002ef0
[2.663] (0) 31 0x0000f2a4:0x00003728
```

After executing SCP_COREDUMP file via `coredump_cmd.sh`, a readable text file will be generated, `analysis.txt`. The file list the combo of function call address, function name and file path of source file.

```
C0 0: 0x0000371a: scp_ipi_queue_init at alps-dev-r0_mp3/vendor/.../adsp_ipi_queue.c:522
C0 1: 0x00008fd6: xQueueSemaphoreTake at alps-dev-r0_mp3/vendor/.../queue.c:1599 (discriminator 6)
C0 2: 0x00008fdc: scp_ipi_queue_init at alps-dev-r0_mp3/vendor/.../adsp_ipi_queue.c:500
C0 3: 0x0000371e: scp_sleep_ctrl at alps-dev-r0 mp3/vendor/.../sleep.c:377
```

8.6 CoreDump

If AP is aware that SCP is not responding to IPI or SCP WDT event, the coredump flow will start automatically. Coredump is a snapshot of the SCP memory and the processor registers like program counter, stack pointer, return address will be saved as well. And much of the system information like system registers, cache contents will be saved as much as possible. This information make it possible to restore the status of system before fault happens.

By default, LLDB is offered to support core dump debug. For LLDB details, please refer to <https://lldb.llvm.org/>.

- LLDB path
 - LLDB can be found in alps prebuilts folder, path `prebuilts/clang/md32rv/linux-x86/lldb_v3`.
- Get Core dump
 - The SCP core dump will be named as “SYS_SCP_DUMP”, which will inside SCP EE DB (e.g., `db.00.EE.dbg`).
 - If not found, please check the EE DB, confirm the `__exp_main.txt`, the Exception type should be “scp”.

```
$ prebuilts/clang/md32rv/linux-x86/lldb_v3/coredump_cmd.sh mt6897 tinysys-scp-RV55_A.elf
SCP_COREDUMP 0
```

```
(lldb) command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/freertos.py
(lldb) freertos

FreeRTOS Awareness is working ...

CPU 0 -- OS Tick: 70125
CPU 1 -- OS Tick: 0
OS is in normal state (neither in Critical Section nor in HW ISR)
TASK-0: IDLE@cpu0 ( TCB: 0x00042c60, State: Running, TCB Number: 8, Priority: 0)
Stack (unit in word) size=1024, current used=124, max used=197
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf mrv_coredump(epc=-61168, regs=0xffff1000)
at exception.c:1055
frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
...
Flight Recorder is working ...
No Flight Recorder Support
(lldb)
```

Note:

LLDB version must be compatible with SCP core.

RV55: lldb_v3

RV33: lldb

If LLDB tool were launched unexpected,

1. Please confirm the file structure under `prebuilts/clang/md32rv/linux-x86/lldb_v3` has not been modified.
2. Please check the related log file `debug_prosim.log` and `debug_ocrd.log`. If the `libprofile.so.x.x.x` libraries were not found, add the following command to `coredump_cmd.sh` before PROSIM launches command.

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$MRV_PDK_PROSIM_HOME
```

```
#add above this command
$PROSIM_EXE $PROSIM_OPT
```

3. There are three tools (LLDB, Openocd, PROSIM) launched when using `coredump_cmd.sh` and tools communicate each other via TCP/IP protocol. If using LLDB-related tools remotely, please ensure that the TCP/IP connection ports are not blocked by the firewall..

8.6.1 LLDB Basic Commands

For the advanced developers or developers who are familiar with GDB, please refer to [command map](#) for the details.

- Back trace: bt

```
(lldb) bt
* thread #1, name = 'Hart0', stop reason = instruction step into
  * frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrvcoredump(epc=-61168, regs=0xffff1000)
  at exception.c:1055
    frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
  prevVal=238376, newVal=<unavailable>) at atomic.c:181
    frame #2: 0x0000befa tinysys-scp-RV55_A.elf`osSetCurrentTid [inlined]
  osSetCurrentTask(task=0x0003a328) at seos.c:153
    frame #3: 0x0000bee6 tinysys-scp-RV55_A.elf`osSetCurrentTid(tid=<unavailable>) at
  seos.c:212
    frame #4: 0x0000d872 tinysys-scp-RV55_A.elf`timFireAsNeededAndUpdateAlarms at
  timer.c:145
    frame #5: 0x0000d608 tinysys-scp-RV55_A.elf`timTimerSetEx(length=<unavailable>,
  jitterPpm=<unavailable>, driftPpm=<unavailable>, info=<unavailable>, data=<unavailable>,
  oneShot=<unavailable>) at timer.c:179
    frame #6: 0x0000cf4e tinysys-scp-RV55_A.elf`osDefer(callback=<unavailable>,
  cookie=<unavailable>, urgent=<unavailable>) at seos.c:1184
    frame #7: 0x000467f4 tinysys-scp-RV55_A.elf`ucHeap + 43796
```

- Dump OS tasks information: freertos

```
(lldb) freertos
Find 18 tasks
CPU 0 -- OS Tick: 70125
CPU 1 -- OS Tick: 0
Coredump cpu: 0
OS is in normal state (neither in Critical Section nor in HW ISR)

TASK-0: IDLE@cpu0 ( TCB: 0x00042c60, State: Running, TCB Number: 8, Priority: 0)
Stack (unit in word) size=1024, current used=124, max used=197
* frame #0: 0x000231c6 tinysys-scp-RV55_A.elf`mrv_coredump(epc=-61168, regs=0xffff1000)
at exception.c:1055
    frame #1: 0x00003eb0 tinysys-scp-RV55_A.elf`atomicCmpXchg32bits(word=<unavailable>,
prevVal=238376, newVal=<unavailable>) at atomic.c:181
    frame #2: 0x0000befa tinysys-scp-RV55_A.elf`osSetCurrentTid [inlined]
osSetCurrentTask(task=0x0003a328) at seos.c:153
    frame #3: 0x0000bee6 tinysys-scp-RV55_A.elf`osSetCurrentTid(tid=<unavailable>) at
seos.c:212
    frame #4: 0x0000d872 tinysys-scp-RV55_A.elf`timFireAsNeededAndUpdateAlarms at
timer.c:145
    frame #5: 0x0000d608 tinysys-scp-RV55_A.elf`timTimerSetEx(length=<unavailable>,
jitterPpm=<unavailable>, driftPpm=<unavailable>, info=<unavailable>, data=<unavailable>,
oneShot=<unavailable>) at timer.c:179
    frame #6: 0x0000cf4e tinysys-scp-RV55_A.elf`osDefer(callback=<unavailable>,
cookie=<unavailable>, urgent=<unavailable>) at seos.c:1184
    frame #7: 0x000467f4 tinysys-scp-RV55_A.elf`ucHeap + 43796
...

TASK-17: CHRE@cpu0 ( TCB: 0x0003f910, State: Suspended, TCB Number: 4, Priority: 4)
Stack (unit in word) size=1024, current used=132, max used=364
* frame #0: 0x00013bc4 tinysys-scp-RV55_A.elf`vTaskExitCritical at tasks_smp.c:4287
    frame #1: 0x00007aee tinysys-scp-RV55_A.elf`evtQueueDequeue(q=<unavailable>,
evtTypeP=<unavailable>, evtDataP=<unavailable>, evtFreeDataP=<unavailable>,
sleepIfNone=<unavailable>) at eventQ.c:171
    frame #2: 0x0000cbb8 tinysys-scp-RV55_A.elf`osMainDequeueLoop at seos.c:1052
    frame #3: 0x0000cdfc tinysys-scp-RV55_A.elf`osMain at seos.c:1093
```

- Print variable and address: p \$(variable name)

```
(lldb) p mTask
(alsPsTask) $0 = {
    id = 259
    handle = ([0] = 16973827, [1] = 16973828, [2] = 16973829)
    prevRtcTime = 0
    mDataSlab = 0x0003c340
    dataEvt = 0x00000000
    alsLastSample = 0
    psLastSample = 1
    bcRecv = {
        [0] = {
            list = {
                prev = 0x00048290
                next = 0x002109e0
            }
        }
        sensor_type = '\f'
        receive_event = 0x00206a34 (tinysys-scp-RV33_A.elf`alsPsReceiveEvent at alsps.c:908)
    }
}
```

- Dump memory: x/FMT address

```
(lldb) x/32xw 0x1ef08
0x0001ef08: 0x003a9593 0x093d3533 0x10063667 0x00bb0ab3
0x0001ef18: 0x01340633 0x016ab5b3 0x00b60c33 0x008c0463
0x0001ef28: 0x008c35b3 0x01248633 0x0433955e 0x35b300b6
0x0001ef38: 0x36330096 0x952e00c4 0x00c50bb3 0xd5334d32
0x0001ef48: 0xc11d094b 0x001ad593 0x001af513 0xfe1c05b3
0x0001ef58: 0x001c5c13 0xfe140c33 0x84338005 0xdb93fe1b
0x0001ef68: 0xeab3001b 0x088500a5 0x652144a2 0x09f4a4b3
0x0001ef78: 0xcf63157d 0x053700a8 0x24237fff 0x8d451001
```

- Read CPU register: Register read \$REG

```
(lldb) register read pc
pc = 0x00000038 tinysys-scp-RV33_A.elf`__divtf3 + 54 at divtf3.c
(lldb) register read
general:
    x0 = 0x00000000 tinysys-scp-RV33_A.elf`vPortInitialiseBlocks at heap_2.c:239
    x1 = 0x0000a32c tinysys-scp-RV33_A.elf`stackDump + 614 at scp_it.c:128
    ...
    x27 = 0x00000000 tinysys-scp-RV33_A.elf`vPortInitialiseBlocks at heap_2.c:239
    x28 = 0x0000006d tinysys-scp-RV33_A.elf`__divtf3 + 107 at divtf3.c:30
(lldb) register read mepc
mepc = 0x0000a32c tinysys-scp-RV33_A.elf`stackDump + 614 at scp_it.c:128
```

8.7 Performance Evaluation and Flight Record

PBFR is a profiling tool which can monitor CPU usage. It can be divided into two sections, performance budget and flight record.

Performance budget contains information with each task loading, cache miss, stall, and the whole system loading. And flight record can record CPU trace during a period.

8.7.1 Performance Budget

Here will introduce how to use performance budget to monitor task loading.

- Enable Performance Budget:
 - Set configuration and definitions to enable performance budget.
 - Path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/inc/FreeRTOSConfig.h
 vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/ mt6897/platform/platform.mk

Table 8-3. Compiler options and definitions

#	Compiler Options and Definitions	Description
1	CFG_PBFR_SUPPORT	PBFR main function control option Enable: Yes Disable: No
2	#define PBFR_SUPPORT_POLLING	Support saving task max loading by polling
3	#define PBFR_SUPPORT_POLLING_MS 10	Polling task activity per 10ms
4	#define PBFR_SUPPORT_CACHE_COUNT	Support cache miss rate
5	#define PBFR_SUPPORT_IOSTALL	Support IO stall rate

- Usage1:
 - Per 10 sec report Performance Budget
 - CFG_MONITORTASK_ALWAYS_RUN = yes
- Usage2:
 - Command: (start/report/end)

```
$ echo 666 0 1 > sys/class/misc/scp/scpctl
```

- Enter the command 1st time to start recording performance budget information, in a period of time; enter the command 2nd time to report the loading information, and enter the command 3rd time to stop recording.
- For some customized requests, these three APIs can be added to specific place for monitoring partial jobs.
 - pbfr_start_loadinfo(0): Start to record loading information
 - pbfr_report_loadinfo(0): Report the loading information
 - pbfr_stop_loadinfo(0): Stop to record loading information

- Log interpretation:

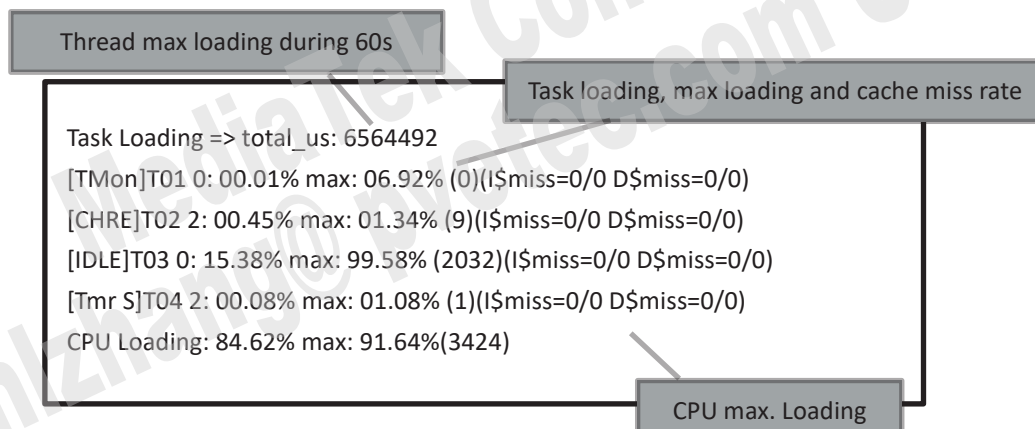


Figure 8-5. Performance Budget log example

8.7.2 Flight Record

Flight Record is a debug tool can record the last events when the system crashes. Here will introduce how to use flight record to monitor CPU traces.

- Enable Flight Record:
 - Set configuration and turn on definitions to get flight record. The configuration in platform.mk is to enable Flight Record, and the definitions in FreeRTOSConfig.h means the events which can be recorded.

- Path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/platform.mk

Table 8-4. Compiler options

#	Compiler Option	Description
1	CFG_PBFR_SUPPORT	PBFR main function control option Enable: Yes Disable: No

- Path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/inc/FreeRTOSConfig.h

Table 8-5. Definitions of Flight Record

#	Definitions	Description
1	#define PBFR_SUPPORT_FLIGHT_REC	Support flight record
2	#define PBFR_MAX_REC_EVENTS 32	Support 32 trace event (maximum: 255)
3	#define PBFR_SUPPORT_REC_TASK	Support task trace events
4	#define PBFR_SUPPORT_REC_QUEUE	Support queue trace events
5	#define PBFR_SUPPORT_REC_SWTIMER	Support SW timer trace events
6	#define PBFR_SUPPORT_REC_INT	Support critical sections trace events
7	#define configTRACE_INT_MASK_TIME_BOUD_NS	Recording critical section trace event only if duration overs time bound
8	#define PBFR_SUPPORT_REC_ISR	Support interrupt service routine trace events
9	#define PBFR_SUPPORT_REC_OSTICK	Support OS tick trace events

- If build failed out of memory size, change the setting in Setting.ini.
 - Log: SCP: {XXX}(size_A>size_B) is out of memory limitation, change the size of XXX (size_B) larger than size_A.

- Path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/Setting.ini

- Usage:
 - Copy tinysys-scp-RV55_A.elf and SCP coredump to the analysis folder.
 - Analysis folder: alps/prebuilts/clang/md32rv/linux-x86/lldb_v3
 - Parsing data with coredump analyzing tool to get SCP_debug.txt and flrec.json.
 - If succussed, it will show the following log.

```
(lldb)command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/flrec.py
(lldb) flrec
Flight Recorder is working ...
Flight Record output done.
(lldb) command script import
/proj/mtk11261/misc/coredump/scp_coredump_cmdline/bin/coredump/fflrec.py
(lldb) fflrec
Flight Recorder is working ...
('xTickCount:', 82801L)
len=1000 status=00000003 RP=123 WP=123 last=82809147582
virtual ns = 82320378580
Flight Record output done.
(lldb)
```

Load flrec.json in chrome://tracing/ to get the GUI result.

- Log interpretation:
 - Console Mode:
 - Console mode will present in SCP_debug.txt, it shows the recorded events before system crashes, and displays events classified by the type. Record[X] indicates that this event is the Xth to last event.

Timestamp (us) for the event

Scheduler switches to the task from previous state

```
[Task events(num=27)]
Record[7]32.182486230: Switch to task14:DEPUTY from a suspended task
Record[18]32.201575461: task13:PRINCIPAL wakeup
Record[20]32.201580076: Switch to task13:PRINCIPAL from a running task
Record[47]32.201730538: task14:DEPUTY wakeup
...
[Queue events:(num=64)]
Record[0]32.182447769: task13:PRINCIPAL Lock => Mutex25 (amount=0)
Record[4]32.182466230: task13:PRINCIPAL UnLock => Mutex25 (amount=1)
Record[22]32.201592384: task13:PRINCIPAL Lock => Mutex25 (amount=0)
...
[Interrupt mask from task (critical section) or ISR]
Record[1] 32.182449922 : Task critical section time 281.6 us > 100 us
Record[2] 32.182453076 : Task critical section time 124 us > 100 us
Record[3] 32.182463845 : Task critical section time 124 us > 100 us
```

Shows critical section events which duration are over than time bound

Figure 8-6. Flight record console mode example

- GUI Mode:
 - Load step
 1. GUI Website: chrome://tracing/
 2. Load json: flrec.json (created in the Analysis folder)
- Instruction of use
 - Use keyboard **W/S** to room in/out, **A/D** to move left/right on the Website, or switch button to change mouse mode.

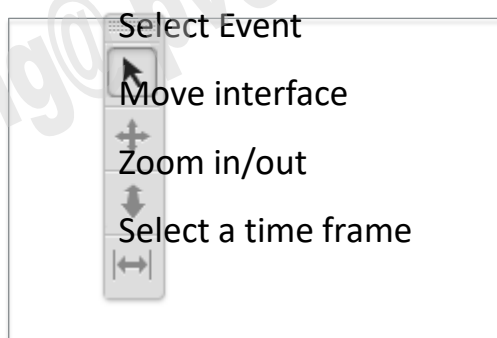


Figure 8-7. Button instruction

- Overall
 - The GUI shows the last events flow, it records six types of events: task, queue event, software timer, critical section, ISR event, and OS tick.

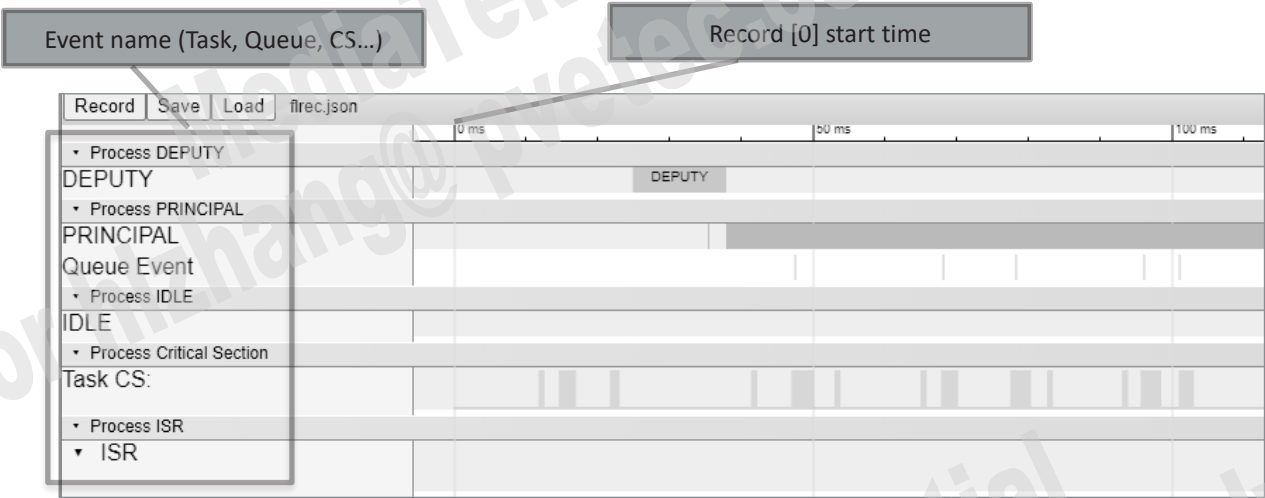


Figure 8-8. Flight record GUI mode interface

The message on the bottom shows the event's start time and duration.

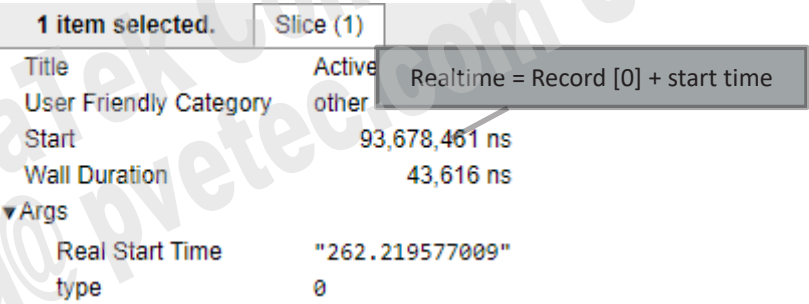


Figure 8-9. Detailed information

- Task
 - The Task event shows the task's wakeup time and execution time.

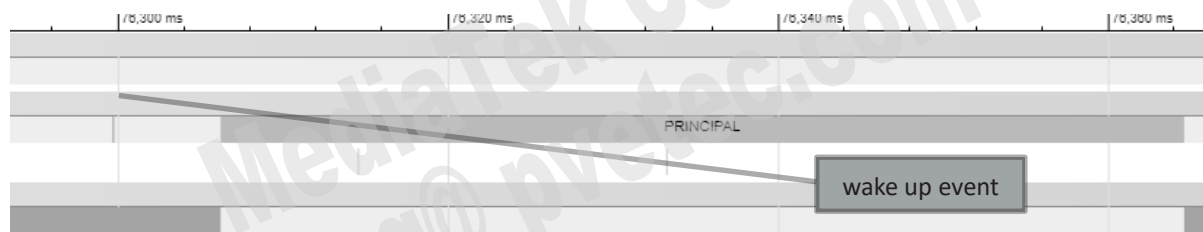


Figure 8-10. Task wake up event

- Queue
 - The Queue event is presented below the task which executes the event and clicking the event, you can see the type of queue showing in the information bar.

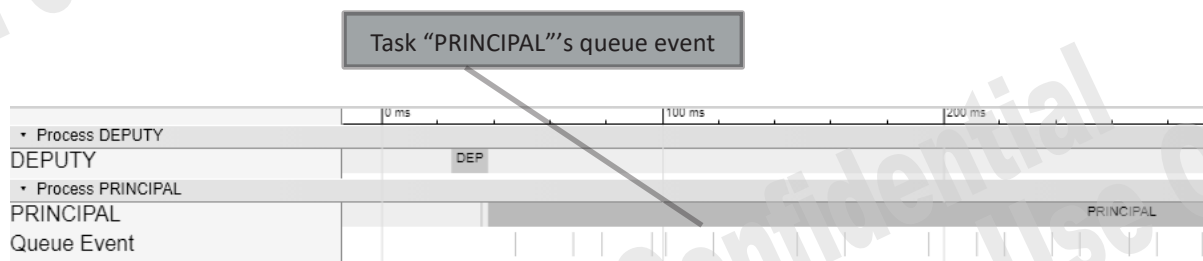


Figure 8-11. Example of Queue event

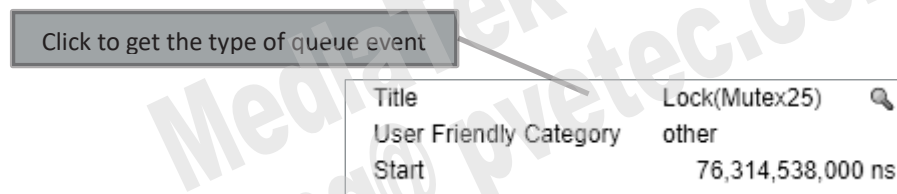


Figure 8-12. Detailed information of queue event

- Software timer
 - SW timer shows the execution time of callback function.

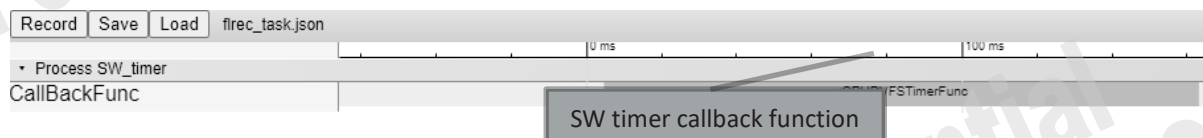


Figure 8-13. Example of SW timer

- Critical Section
 - By moving upward through the corresponding critical section, you can find the tasks running in the critical section.

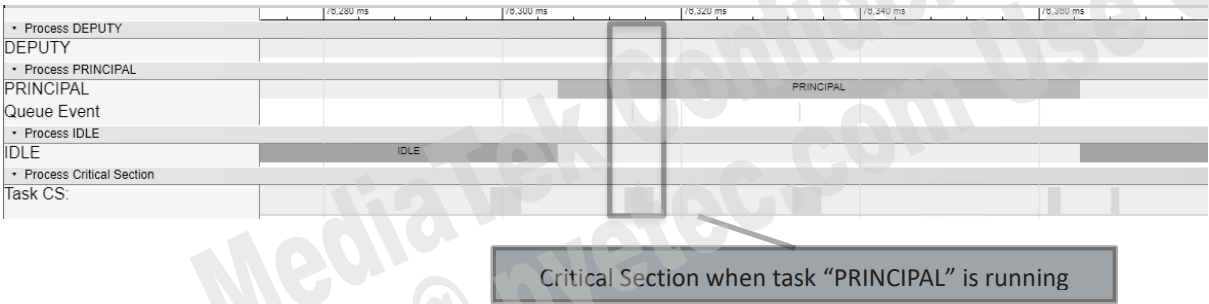


Figure 8-14. Example of critical section

- ISR
 - ISR shows the execution time of interrupt service routine and ISR number.

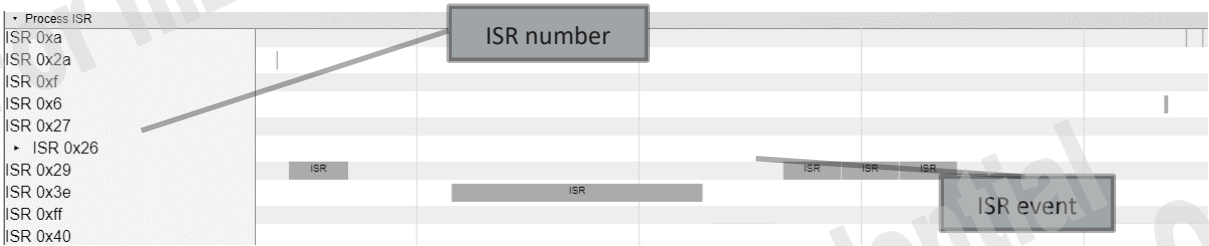


Figure 8-15. Example of ISR

- OS Tick
 - OS tick displays the time of the operating system timer ticks.

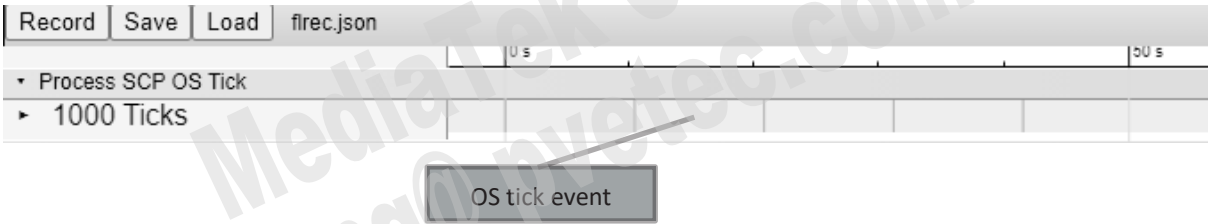


Figure 8-16. Example of OS tick

9 Address Sanitizer

9.1.1 ASAN Usage

Address Sanitizer (ASAN) is an address sanity checker to find out-of-bounds accesses to heap, stack, and global objects, as well as use-after-free bugs.

- Enable ASAN
 - Set configuration and turn on definitions to enable ASAN.
 - Path

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/\$Project/project.mk
 vendor/mediatek/proprietary/tinysys/scp/build/config.mk

Table 9-1. Compiler options and definitions

#	Compiler Options and Definitions	Description
1	CFG_ASAN_SUPPORT	Enable: Yes Disable: No
2	-mllvm -asan-stack	Check local variable Enable: 1 Disable: 0
3	-mllvm -asan-global	Check global variable Enable: 1 Disable: 0
4	-mllvm -asan-memintrin	Check memset/memecpy Enable: 1 Disable: 0

- Exception

Table 9-2. Exception type

#	Exception Type	Description
1	ASAN_STACK_LEFT	Local variable underflow
2	ASAN_STACK_MID	Local variable underflow/overflow
3	ASAN_STACK_RIGHT	Local variable overflow
4	ASAN_STACK_PARTIAL	Local variable overflow
5	ASAN_GLOBAL_REDZONE	Global variable overflow
6	ASAN_HEAP_USE_AFTER_FREE	Heap use after free

- Issue handling - Stack underflow

```
[14.504] (0) [0] KASAN detect stack-out-of-bounds at PC:0x15f52
[14.505] (0) [0] write 1Byte at [0x3c298] -> shadow [0x207853]
[14.506] (0) [0] Shadow bytes around the buggy address:
[14.507] (0) [0] 0x00207830: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.508] (0) [0] 0x00207840: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.509] (0) [0] =>0x00207850: f1 f1 04 [f2] 00 02 f3 f3 00 00 00 00 00 00 00 00
[14.510] (0) [0] 0x00207860: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.512] (0) [0] 0x00207870: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[14.513] (0) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 144
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
```

```
00015f42 <asan_underflow_left>:
; asan_underflow_left():
; /mfs/mtk51157/mtk26942/casws/alps-mp-t0_mp1-of-p8_b/vendor/
; {
15f42: 9005      push16.qw.m0
15f44: 717d      c.addi16sp sp, -16
; /mfs/mtk51157/mtk26942/casws/alps-mp-t0_mp1-of-p8_b/vendor/
; ptr[-1] = 0xAA;
15f46: fff50493  addi    s1, a0, -1 // 0xffff [12bitSE]
15f4a: 842a      c.mv    s0, a0
15f4c: 8526      c.mv    a0, s1
15f4e: 8adf40ef  jal    -46932 <__asan_store1>
15f52: 0aa00513  li     a0, 170
```

```
__attribute__((noinline)) void asan_underflow_left(char *ptr)
{
    ptr[-1] = 0xAA;
    printf("%s: underflow write %p\n", func_, &ptr[-1]);
}
```

- Issue handling - Global variable overflow

```
[8.490] (0) [0] KASAN detect global-out-of-bounds at PC:0x15f64
[8.491] (0) [0] read 4Byte at [0x60288] -> shadow [0x20c051]
[8.492] (0) [0] Shadow bytes around the buggy address:
[8.493] (0) [0] 0x0020c030: f9 f9 f9 f9 00 f9 f9 f9 f9 f9 04 f9 f9 f9 f9
[8.494] (0) [0] 0x0020c040: f9 f9 f9 00 f9 f9 f9 f9 f9 f9 00 00 00 00 00
[8.495] (0) [0] =>0x0020c050: 00 [f9] f9 f9 f9 f9 f9 04 f9 f9 f9 f9 f9 f9
[8.496] (0) [0] 0x0020c060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[8.498] (0) [0] 0x0020c070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[8.499] (0) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan_helper_v1.c, line: 144
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
```

```
; /mfs/mtk51157/mtk26942/casws/alps-mp-t0_mp1-of-p8_b/vendor/mediatek
test = ptr[10];
15f52: 00060537  lui    a0, 96 // 0x60000
15f56: 26050413  addi    s0, a0, 608 // 0x260 (i32: 0x60260)
15f5a: 02840493  addi    s1, s0, 40 // 0x28
15f5e: 8526      c.mv    a0, s1
15f60: 80bf40ef  jal    -47094 <__asan_load4>
15f64: 854a      c.mv    a0, s2
```

```
__attribute__((noinline)) void global_overflow(void)
{
    volatile uint32_t *ptr;
    volatile uint32_t test;
    ptr = &globalArr[0];
    test = ptr[10];
}
```

- Issue handling - Heap use after free

```
[9.230] (1) [0] KASAN detect heap use-after-free at PC:0x40c2a
[9.231] (1) [0] read 4Byte at [0x271b90] -> shadow [0x54e372]
[9.232] (1) [0] Shadow bytes around the buggy address:
[9.232] (1) [0] 0x0054e350: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[9.233] (1) [0] 0x0054e360: 00 00 00 00 00 00 00 00 00 00 00 00 fd fd fd fd
[9.234] (1) [0] =>0x0054e370: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.235] (1) [0] 0x0054e380: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.236] (1) [0] 0x0054e390: fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd fd
[9.236] (1) [0] [ASSERT] task: Main, file: ../common/middleware/sanitizer/asan/asan helper v1.c, line: 135
```

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Global redzone: f9
Heap use after free: fD
```

```

/mfs/mtkslt1157/mtk26942/casws/alps-mp-t0_mpl-of-p
  if(ptr[5] == 0xAA)
40c22: 01440513    addi    a0, s0, 20 // 0x14
40c26: 873cb0ef    jal    -214926 <_asan_load4>
40c2a: 4848      c.lw    a0, 20(s0)
40c2c: 0aa00593    li     a1, 170
40c30: 00b51863    bne    a0, a1, 16 <.LBB9_2>

```

```
uint32_t *dynamicArr;  
attribute__((noinline)) void asan_use_after_free(void)  
{  
    uint32_t *ptr;  
    dynamicArr = pvPortMalloc(10*sizeof(uint32_t));  
    vPortFree(dynamicArr);  
    ptr = dynamicArr;  
    if(ptr[5] == 0xAA)
```

9.1.2 How to Move Code to DRAM

After enabling ASAN, the code size will increase about 1.7x. Therefore, the SRAM size might be insufficient. The user can move some feature or function to DRAM to reduce the DRAM usage.

- Use attribute

Use Cache macros defined in cache internal to specify function/variable on DRAM region.

Beware, DRAM REGION VARIABLE works only on global/static variables because local variables use stack as their storages.

By default, for the performance concern, stack is preserved in SRAM.

```
#define DRAM_REGION_VARIABLE __attribute__((section (".dram_region_variable")))
#define DRAM_REGION_FUNCTION __attribute__((section (".dram_region_func")))
```

- Use linker script

The linker script describes how objects are mapped in the output binary file.

It provides an easy way to locate objects to DRAM region.

Here template header and linker script are provided to locate all sections in a C file to DRAM region.

1. Check `cache_ld.h` (`@project/RV55_A/$platform/$project`), here you can see lots of predefined macros with style `CACHE_***_TEXT& CACHE_***_DATA`. You can add one new macro and name it by yourself. The string after name is the expected c file path.

- For example: (The “?” is wildcards compare)

```
#define CACHE_BARO_TEXT \
    */middleware/contexthub/MEMS_Driver/barometer/*.o(.text*) \
#define CACHE_BARO_DATA \
    */middleware/contexthub/MEMS_Driver/barometer/*.o(.rodata* .data* .bss*) \
```

2. Check `link.ld`, it should have `CACHE_***_TEXT` & `CACHE_***_DATA`.

- You can add the string after `CACHE_BARO_TEXT/CACHE_BARO_DATA`.
- For example:

```
dram_region __dram_start : AT(__dram_start) {  
#ifdef CACHE_BARO_TEXT  
    CACHE_BARO_TEXT  
#endif  
*(.dram_region_func)  
    *(.dram_region_ro)  
  
#ifdef CACHE_BARO_DATA  
    CACHE_BARO_DATA  
#endif  
}
```

10 Appendix

10.1 Frequently Asked Questions/Troubleshooting

10.1.1 Malloc Failure

Due to the limited and relatively small size of SRAM, the size of the malloc pool is tailored to the functionality. It is common for malloc to fail when introducing new features without expanding the pool size.

Note:

When malloc failure happens, a message will be showed.

```
[2.232] (0) malloc fail
```

```
[2.232] (0) [ASSERT] task: CHRE
```

This message indicates which task experienced a malloc failure, followed by a core dump log.

• How to Fix

1. Modify the `scp/project/RV55_A/mt6897/platform/platform.mk`, enlarge the heap size, e.g., 80×1024

```
ifeq ($(CFG_CHRE_SUPPORT),yes)
$(eval TOTAL_HEAP_SIZE=$(shell echo $$(( $(TOTAL_HEAP_SIZE) + (80 * 1024) ))) ) • change 40
to 80
endif
```

2. Modify the `scp/drivers/common/scpctl/scp_scpctl.c`; enable task monitoring forcibly.

```
void scpctl_init(void)
{
...

    else { /* monitor task is in suspended state */
        scpctl.stat = SCPCTL_STAT_INACTIVE;
        scpctl.op = SCPCTL_OP_INACTIVE;
        //vTaskSuspend(xMonitorTask[id]); • about line: 244, mark it
    }

...
}
```

3. Check the remaining heap size and compute the required heap size

```
[210.009] (0) Heap:free/total:40946/108544
```

4. Correct the heap size, the remaining heap size is 40946, $40946/1024 = 39.98$, the minimum heap size is 41, and setting it to 43 or 44 is safer. Please remember to revert the changes in `scp/drivers/common/scpctl/scp_scpctl.c` after the issue is fixed.

```
ifeq ($(CFG_CHRE_SUPPORT),yes)
$(eval TOTAL_HEAP_SIZE=$(shell echo $$(( $(TOTAL_HEAP_SIZE) + (43 * 1024) ))) )
endif
```

10.1.2 Unaligned Access

The SCP MDSP-RV33 processor does not have hardware to handle unaligned access, and the implementation is done by software. When an unaligned access occurs, the processor raises an exception and handles it through software in the handler. Therefore, there is a significant performance cost.

- **Note:**

When unaligned access happens, a warning message will be showed.

"Warning: MISALIGNED LOAD, pc:0x00001234, addr:0x00042232"

This message shows the PC with problems and address the processor wants to access. The system may be busy to print this message when unaligned access happens continuously and may cause timeout assertion.

- **How to Fix**

Most unaligned accesses are due to the use of the packed qualifier in structures. This can be avoided by removing it.

- **Example:**

```
struct pack_struct {  
    uint32_t size;  
    uint32_t crc;  
    uint8_t type;  
} __attribute__((packed));
```

When a structure array is declared, e.g., `pack_struct st_array[10]`, the access of structure member will be unaligned.

Another case is accessing character array with integer pointer. This can be avoided by adding

`__attribute__((aligned(4)))`. The compiler does not guarantee `&char_array[0]` is 4 byte alignment.

- **Example:**

```
uint8_t char_array[64];  
test_value = *(uint32_t *)(&char_array[0])
```

10.2 Code Size Limitation

The total SRAM size of MT6897 is 2MB, and the actual SCP SRAM size can be determined by checking the symbol `_end`.
The remaining SRAM size::

```
MT6897: 0x300000 - _end
```

In MT6897, SRAM region LENGTH can be set to 2MB in `project/RV55_A/MT6897/platform/link.ld.c`

- Example:
`sram : ORIGIN = 0x00000000, LENGTH = 0x00200000`

When size exceeds 2MB, this will cause build failure.

The code size checking tool will run at each build and output detailed information. Refer to **Error! Reference source not found.** table below, where the vertical axis represents the project names, and the horizontal axis represents the code size for each project. For example, the total size of CHRE is 62985 (sum) bytes, with the .text section accounting for 35598 bytes of the total size.

		. = ALIGN (...sync	.text	Sum
C-lib	0 ...	0	0	
CHRE	0 ...		35598	62985
DSP	0 ...	0	0	
DVFS	0 ...	7574	8606	
Heap	0 ...	334	82275	
Peripheral	0 ...	18064	26981	
Platform	256 ...	66300	313481	
RTOS	0 ...	13776	14536	
Sensor	0 ...	0	0	
VOW	0 ...	0	0	

10.2.1 Code Size Tool Usage

- The tool **memoryReport.py** is a script which is used to limit code size during builds. If the code size exceeds your settings, it will result in a build error.

– Path

```
vendor/mediatek/proprietary/tinysys/common/tools/memoryReport.py
```

- Set the path for configuration file

– Path

```
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/Setting.ini
```

- Configuration file format (setting.ini)

```
[TinySys-SCP]
$File_Name: $Main_feature: $Sub_feature
[SCP-MT6985]
$Main_feature: Max_code_size
$Sub_feature: Max_code_size
```

- * File_name: Full file path or partial file path (e.g., middleware/contexthub/perf)
- * Main feature, (e.g., Sensor, Audio): The main feature that this file belongs to
- * Sub feature, (e.g., gyro, pedometer): The sub feature that this file belongs to
- * Main_feature/Sub_feature (after SCP-MT6985): The maximum size limit of the main or sub

- Memory check fails
 - Example

```
SCP: I2C(3958>110) is out of memory limitation
SCP: SPI(6316>1100) is out of memory limitation
make: *** [tinysys_out/RV55_A/scp/tinysys-scp-RV55_A.elf] Error 13
```

10.3 Scp_Region_Info Structure

The struct `scp_region_info_st` is a pointer to a fixed address at SCP SRAM. It is used to pass parameter from bootloader to SCP before the IPI is ready. Remember to synchronize the bootloader (lk2), kernel (kernel-6.1), and SCP repository code. If the structure is not synchronized between repositories, the SCP may fail to start..

LK header:

vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/mt6897/common/scp/scp_plat_priv.h

Kernel header:

kernel/kernel_device_modules-6.1/drivers/misc/mediatek/scp/rv/scp_helper.h

SCP header:

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/inc/main.h

SCP boot:

vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897 /platform/boot55.S

`scp_region_info` also defined in `boot55.S`.

For example, if a new member called test ID is added in structure `scp_region_info`, the default value must be added in `boot55.S`.

```
scp_region_info:
.long    0x00000000    /* 0x04 ap_loader_start */
.long    0x00000000    /* 0x08 ap_loader_size */
...
.long    0x00100000    /* 0x34 regdump size */
.long    0x00000000    /* 0x38 param start address */
.long    0x00000000    /* 0x3c ap_params_start */
.long    0x12345678    /* 0x40 a test ID */
```

Access `scp_region_info_st` in SCP side, then it must be assigned to `NULLPTR_BASE`, for example

```
void scpctl_init(void)
{
    int ret = 0;
#ifdef CFG_NULLPTR_TRAP
    struct scp_region_info_st region_info = *(struct scp_region_info_st
*) (NULLPTR_BASE + 0x4);
#endif
```

10.4 SCP Recovery

The Linux SCP driver will do SCP recovery when SCP crashes or does not respond. After the recovery process, SCP will be back to normal.

Note:

- When SCP is reset, SCP driver will clear the contents of SCP SRAM/DRAM, reset the RV55 processor in ATF, and re-execute the boot process. (Clearing all program text, bss, and data segments)
- SCP only resets the processor **but not** peripherals (such as sensors, I2C modules, and other devices), so if necessary, peripheral drivers should reset themselves in the initial stage.
- Developers of the Linux SCP driver API **MUST** follow the **Reset notify flow** in [Section 10.4.2](#) and ensure that there is no communication between Linux driver and SCP during recovery.
- When SCP restarts, Linux SCP driver will be reinitialized. Drivers related to the SCP driver must ensure that the restart process does not affect their functionality.

10.4.1 Recovery Behavior

When the SCP is abnormal or unresponsive for a period of time, recovery will start. The SCP Kernel driver and SCP will enter a service interrupt state. During this time, the SCP kernel driver will reset the SCP and send SCP_EVENT_STOP to all registered notify chain drivers. Once the SCP is back to normal, the SCP Kernel driver will send SCP_EVENT_READY to all drivers.

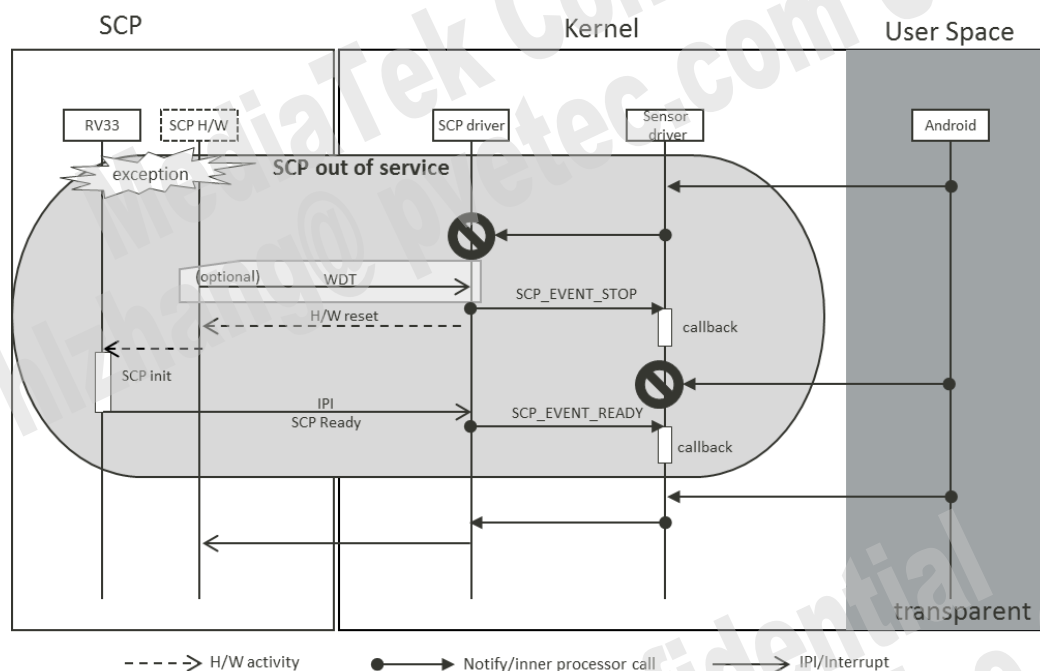


Figure 10-1. SCP recovery behavior

10.4.2 Recovery Notify Flow

Developers of the Linux SCP driver **MUST** follow this SCP EVENT notify chain to ensure effective SCP recovery.

- API:** Receives notifications and invokes registered callback functions

```
void scp_A_register_notify(struct notifier_block *nb)
```

description

To register callback function of notify chain

parameters

nb: callback function of notify chain

callback parameters

SCP_EVENT_READY: when SCP initial done, start tasks corresponding to SCP

SCP_EVENT_STOP: when SCP is going to be reset, stop tasks corresponding to SCP

- **Note:** All callback functions may be called multiple times and **MUST NOT** be blocked.
- **Example**
 - Please include `<linux/notifier.h>`, `<mach/scp_helper.h>`
 - Call `scp_register_notify ()` with argument: `SCP_EVENT_READY` or `SCP_EVENT_STOP`

```
static void task_start(void) {
    // start tasks
}
static void task_stop(void) {
    // stop tasks
}
static int app_event(struct notifier_block *this, unsigned long event, void *ptr) {
    switch (event) {
        case SCP_EVENT_READY:
            task_start();
            break;
        case SCP_EVENT_STOP:
            task_stop();
            break;
    }
    return NOTIFY_DONE;
}
static struct notifier_block app_notifier = {
    .notifier_call = app_event,
};
static int __init scp_app_init(void) {
    if (scp_is_ready()) {
        task_start();
    }
    scp_register_notify(&app_notifier);
}
```

Note:

- Must register notification chain because the `scp_ipi_send()` may return error during recovery.
- An error handling flow must be applied:
 - Stop calling `scp_ipi_send()` right after receiving `SCP_EVENT_READY`
 - Resume after receiving `SCP_EVENT_READY`.

11 Q&A List from Users

11.1 How to Enlarge DRAM Region Code

By default 1MB DRAM is reserved for storing DRAM code. In special case, if more space is needed for storing code, here is the method of modification.

1. Modify LK, enlarge 0x100000

```
vendor/mediatek/proprietary/bootable/bootloader/lk2/platform/mediatek/common/scp/RV/scp_pl
at_priv.h
#ifdef MTK_MINIMUM_SCP_DRAM_SIZE
#define SCP_DRAM_IMG_SIZE      0x080000    // 0.5MB dram image
#else
#define SCP_DRAM_IMG_SIZE      0x100000    // 1.0MB dram image
#endif
```

2. Modify SCP, enlarge 0x00100000

```
vendor/mediatek/proprietary/tinysys/scp/project/RV55_A/mt6897/platform/link.ld.c
MEMORY {
    ... ..
    dram      :   ORIGIN = 0x00200000, LENGTH = 0x00100000
    ... ..
}
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek's product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.